



TECHNICAL DOCUMENTATION

ATCS — Alternative Text-based Communication System

*Architecture, Protocol, and Implementation of an
Offline Emergency Mesh Communication System*

SYSTEM	ESP32 LoRa mesh nodes ("Rangers") + Android/iOS companion app
FIRMWARE	Arduino / C++, SX127x LoRa radio, WebSocket bridge
APPLICATION	Next.js 15 / React 19 / TypeScript, packaged with Capacitor 7
DEPLOYMENT CONTEXT	Field-tested, Dodoma, Tanzania — zero-infrastructure operation
DOCUMENT VERSION	1.0

ABSTRACT

ATCS is a two-tier offline communication system built for environments with no cellular or internet infrastructure. A constellation of battery-powered ESP32

microcontroller nodes ("Rangers"), each fitted with a LoRa long-range radio, forms a self-healing multi-hop mesh network using a reactive, AODV-inspired routing protocol. Each node also runs a local Wi-Fi access point and an embedded WebSocket server, which a companion smartphone application connects to directly — translating simple JSON commands into LoRa mesh traffic and vice versa. On top of this transport, the application layer provides text messaging, an emergency broadcast channel, one-time and continuous location sharing, and compass-based peer-to-peer navigation, all of which continue to function with absolutely no external network reachability at any point in the stack.

This document describes the system end-to-end: its network topology and protocol design, the firmware's mesh-routing implementation, the mobile application's architecture and state model, and the cross-cutting engineering problems encountered while integrating the two — including the rationale behind design choices that were tried, revised, or discarded along the way.

Table of Contents

1 Executive Summary

2 System Overview

2.1 Problem Statement and Motivation

2.2 High-Level Architecture

2.3 Design Philosophy

2.4 Technology Stack

3 Network and Communication Topology

3.1 Physical Topology

3.2 Logical Layering

3.3 Addressing Scheme

4 ESP32 Firmware Architecture

4.1 Hardware Platform and Radio Configuration

4.2 Program Structure

4.3 Concurrency Model — Design Decision

5 Mesh Networking and Routing Protocol

5.1 Packet Format

5.2 Packet Types

5.3 Neighbor Discovery — HELLO Beacons

5.4 Reactive Routing — RREQ / RREP

5.5 Forwarding and Flooding

5.6 Duplicate Suppression

5.7 Reliability — ACK, Retry, NACK

5.8 Outgoing Queue and Priority Arbitration

5.9 Routing Design Alternatives Considered

6 Communication Protocol

6.1 Transport Choice — WebSocket over Wi-Fi

6.2 Firmware → Phone Messages

6.3 Phone → Firmware Messages

6.4 Application-Layer Sentinels

6.5 Liveness and Heartbeats

6.6 Worked Example — A Message's Full Journey

7 Messaging Subsystem

7.1 Send Flow and Delivery State Machine

7.2 Receive Flow and Deduplication

7.3 Offline Queueing

8 Reachability and Signal Quality

9 Navigation and Compass Subsystem

9.1 From Offline Maps to Compass-Only — A Pivot

9.2 Bearing and Distance Mathematics

9.3 Sensor Fusion and Calibration

9.4 Smoothing Pipeline

9.5 Navigation Session Lifecycle

10 Location Sharing

10.1 One-Time Request / Response

10.2 Continuous Live Sharing

11 Emergency (SOS) Subsystem

12 Mobile Application Architecture

12.1 Platform Choice — Web-Native Hybrid

12.2 Component and State Architecture

12.3 Connection Lifecycle and Resilience

13 Cross-Cutting Engineering Case Studies

13.1 The Triple Keyboard-Compensation Bug

13.2 Notification Stacking and Edge-to-Edge Layout

14 End-to-End System Walkthrough

15 Design Trade-offs Summary

16 Limitations and Future Work

17 Conclusion

A Appendix — Firmware Constants Reference

B Appendix — WebSocket JSON Schema Reference

Executive Summary

ATCS (Alternative Text-based Communication System) is an offline-first emergency communication platform designed to let people coordinate, share their location, and signal distress in areas with no cellular coverage and no internet access — dense forest, mountainous terrain, disaster zones, or any region where conventional infrastructure has failed or never existed. The system was field-tested in Dodoma, Tanzania, under realistic conditions: real GPS drift, real LoRa range limits, and real hardware constraints.

The architecture rests on two cooperating tiers. The first is a **hardware mesh tier**: a set of ESP32 microcontrollers, each paired with a LoRa long-range radio module, that relay packets for one another using a lightweight, reactive routing protocol modelled on AODV (Ad hoc On-Demand Distance Vector routing). Nodes require no fixed infrastructure — no router, no base station, no SIM card — and a message can travel several hops across nodes that never directly hear one another. The second is a **companion application tier**: a cross-platform mobile app built with Next.js and packaged natively with Capacitor, which never talks to the mesh directly. Instead, each phone joins the Wi-Fi access point broadcast by whichever Ranger node is nearest, and speaks a small JSON protocol to that node's embedded WebSocket server. The node then acts as the phone's sole gateway into the LoRa mesh.

On top of this transport the application layer implements five user-facing capabilities: private and broadcast text messaging with delivery confirmation and automatic retry; a dedicated Emergency broadcast channel with mesh-wide SOS propagation and per-node confirmation; one-time and continuous ("live") location sharing between consenting peers; compass-based bearing-and-distance navigation toward another person's last known position; and device discovery with an automatic pairing handshake — all of it functioning identically whether two nodes are in the same room or three hops apart, and all of it degrading gracefully, never silently, when a link is lost.

This document is organised to first establish the system's shape (Sections 2–3), then descend into each subsystem in turn — firmware, mesh protocol, application protocol, messaging, navigation, location sharing, emergency handling, and the mobile application's own architecture (Sections 4–13) — before reassembling the

pieces into full end-to-end walkthroughs and a candid account of the trade-offs made along the way (Sections 14–17).

System Overview

2.1 Problem Statement and Motivation

Conventional communication — cellular voice/data, Wi-Fi backed by an ISP, satellite terminals — all depend on infrastructure that is either absent by geography (remote wilderness, mountainous terrain) or destroyed by circumstance (earthquake, flood, conflict). In these conditions the people who most need to coordinate — search-and-rescue teams, field workers, hikers, disaster responders — are left with line-of-sight radio, if anything at all. LoRa (Long Range) radio modulation offers kilometre-scale range at milliwatt power budgets, but a single LoRa link is still just a link: two devices out of each other's radio range cannot talk unless something in between relays for them. ATCS's core engineering problem is therefore twofold: (1) build a self-organising multi-hop network out of cheap, battery-powered LoRa nodes with no central coordinator, and (2) present that network to an ordinary smartphone user as a familiar messaging-and-location experience, without ever touching the internet.

2.2 High-Level Architecture

Figure 2.1 shows the two-tier shape of the system for a three-node deployment. Each Ranger node is simultaneously (a) a LoRa mesh participant, relaying packets for other nodes, and (b) a local Wi-Fi access point with an embedded WebSocket server that exactly one nearby phone connects to. A phone never speaks LoRa directly and never needs to know the mesh topology — it issues simple commands ("send this text to Node 3") to its own node and lets the mesh figure out how to get it there.

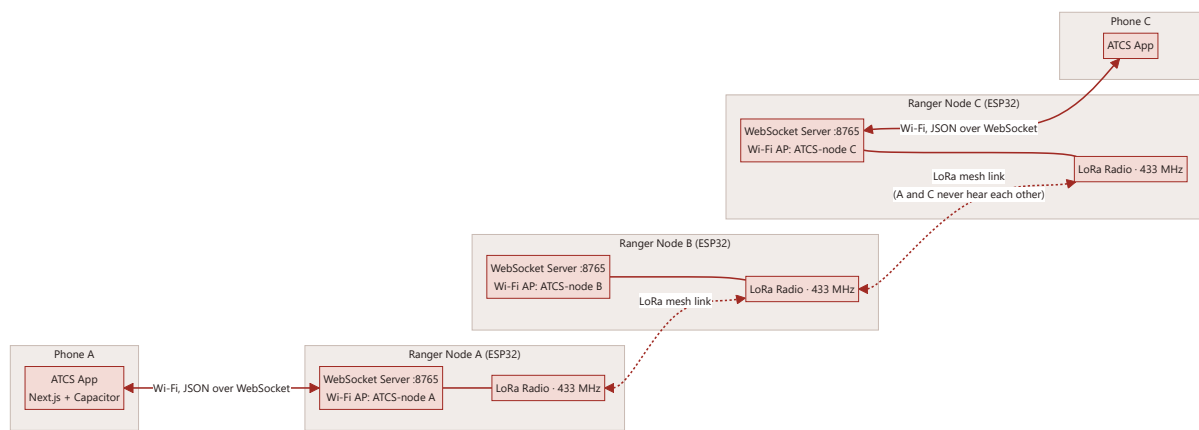


Figure 2.1 — Two-tier system architecture. Phone A reaches Phone C by relaying through Node B, even though Node A and Node C are out of direct radio range of one another. Neither phone is aware this relay happened; it is transparent below the application protocol.

Both tiers are deliberately decoupled: the firmware has no concept of "chat," "contact," or "delivery status" — it only understands addressed byte packets, acknowledgements, and routes. Every higher-level concept (a conversation thread, a delivery checkmark, a contact list) is an application-layer construct built entirely inside the phone app on top of a transport that only promises "best-effort, retried, addressed packet delivery." This separation is what allows the mesh protocol to stay small and auditable while the user-facing feature set keeps growing.

2.3 Design Philosophy

- **Zero infrastructure, zero internet, by construction.** Every code path that could depend on internet reachability (e.g. an IP-geolocation fallback, discussed in Section 9.3) is explicitly gated behind an opt-in flag that defaults to *off*, rather than being trusted to fail gracefully. A feature that silently assumes connectivity is treated as a defect, not a convenience.
- **Fail loud, not silent.** A message that cannot be delivered is reported as failed, not left in limbo; a contact that has gone quiet is shown as stale or offline on a fixed schedule, never left showing a confident "online" indefinitely. Section 8 details the two independent classifiers that enforce this.
- **Best-effort transport, reliable-feeling application.** LoRa packets can be lost, corrupted, or collide; the mesh layer compensates with acknowledgements, bounded retry, and route rediscovery (Section 5.7) so that the application layer

can present ordinary "sent → delivered" status semantics to the user without reinventing reliability itself.

- **Small, fixed-size, army of specialised tables over one generic structure.** The firmware favours several small, fixed-capacity arrays (a route table, a neighbor table, a duplicate cache, a pending-ACK table) over a single dynamic data structure — deliberate, given an ESP32's limited RAM and the value of predictable, bounded memory use in a device that may run unattended for days.

2.4 Technology Stack

Layer	Technology	Role
Radio hardware	SX127x-family LoRa transceiver (Arduino LoRa library)	Physical-layer long-range, low-power radio link
Node MCU	ESP32 ("Rola" board variant), Arduino/C++ firmware	Mesh routing, Wi-Fi AP, WebSocket bridge, LED/button UX
Node ↔ phone transport	Wi-Fi (802.11, AP mode) + WebSocket (port 8765) + JSON (ArduinoJson)	Bridges the LoRa mesh to a locally-connected phone
Application framework	Next.js 15 (static export) / React 19 / TypeScript	Cross-platform UI and business logic, compiled to static assets
Native packaging	Capacitor 7 (Android target: minSdk 23 / targetSdk 35)	Wraps the web app in a native WebView shell with plugin access to GPS, keyboard, network state, and app lifecycle
Offline app resilience	Servist (Workbox-based service worker)	PWA install, background sync for offline-queued messages
UI layer	Tailwind CSS v4, Radix UI primitives, Lucide icons	Component styling and accessibility primitives

Network and Communication Topology

3.1 Physical Topology

Each Ranger node operates its Wi-Fi radio in Access Point mode (`WiFi.mode(WIFI_AP)`), broadcasting its own SSID (e.g. `ATCS-node 4`) with a fixed password. This is a deliberate single-hop-only design decision for the phone-to-node link: a phone always joins exactly one node's hotspot at a time, and that node is its only gateway into the mesh. The LoRa radios, in contrast, form an arbitrary *ad hoc* mesh — any two nodes within radio range of each other can hear one another directly, and nodes outside each other's range communicate only through intermediate relays. Figure 3.1 illustrates a five-node deployment where no single node can reach every other node directly.

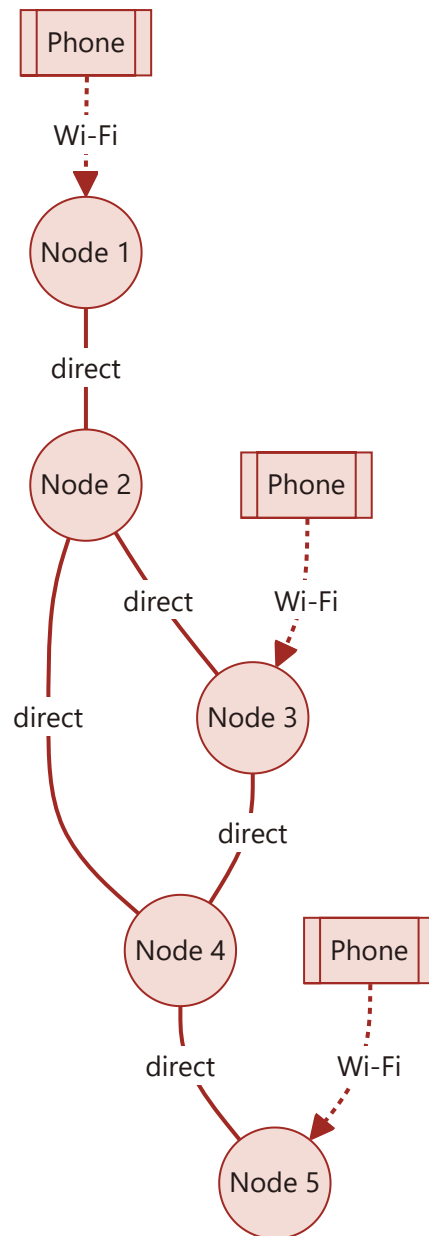


Figure 3.1 — A five-node mesh with partial connectivity. Node 1 and Node 5 cannot hear one another directly; a message between the phones attached to them must be relayed through at least two intermediate nodes (e.g. $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$, or $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$), and the routing layer (Section 5.4) discovers whichever path actually works without either phone knowing the topology exists.

3.2 Logical Layering

The system is best understood as four stacked layers, each with a narrow, well-defined contract to the layer above it. No layer reaches past its immediate neighbour: the application never encodes a raw LoRa packet, and the firmware never parses

application semantics out of a payload — payloads are opaque bytes to the mesh layer, full stop.

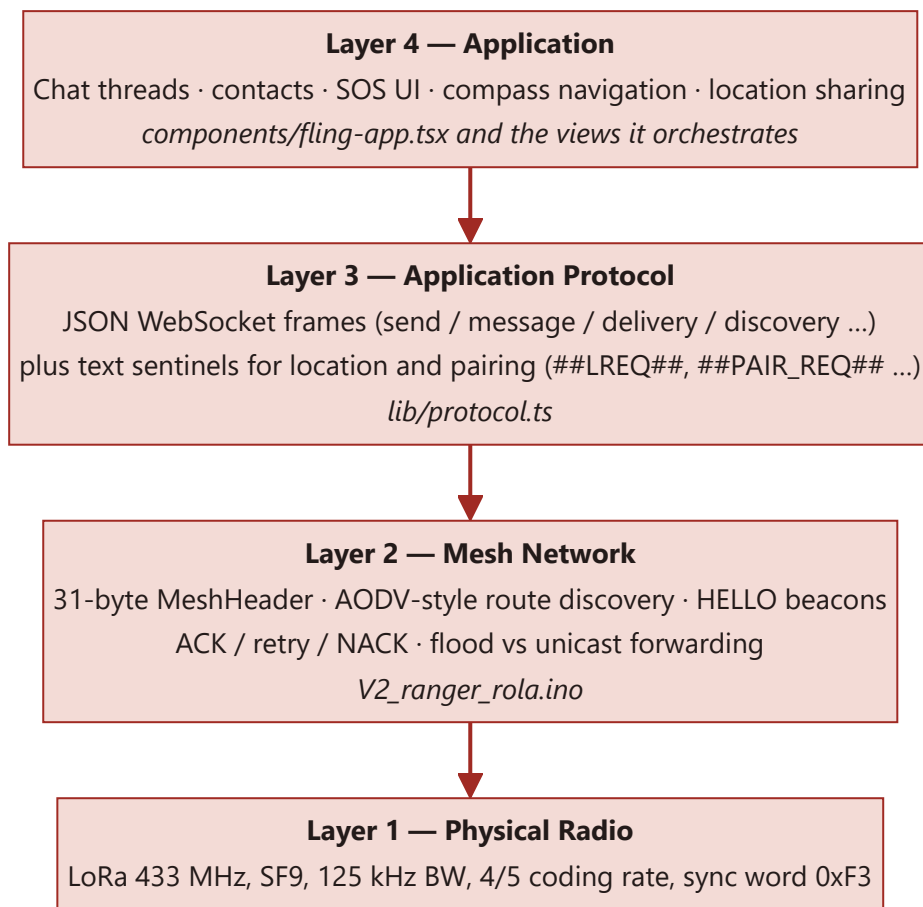


Figure 3.2 — Logical protocol stack. Layer 3 and 4 both run entirely on the phone; only Layer 2 and Layer 1 run on the ESP32. The WebSocket link in Layer 3 is the sole crossing point between the phone's process and the node's firmware.

3.3 Addressing Scheme

Every node has a fixed, human-assigned identifier — an ASCII string up to 8 characters (`NODE_ID_LEN = 8`), such as `Node4` — configured at flash time via a `#define` in the firmware and never negotiated or auto-assigned. This keeps addressing trivial to reason about at the cost of manual configuration per device: two nodes flashed with the same ID would be indistinguishable to the mesh, so ID uniqueness is an operational, not a protocol-enforced, invariant. The reserved address `"*"` means *broadcast* — a packet addressed to `"*"` is delivered to every

node that physically receives it and, if its TTL allows, re-flooded outward, giving the system its Emergency broadcast capability described in Section 11.

ESP32 Firmware Architecture

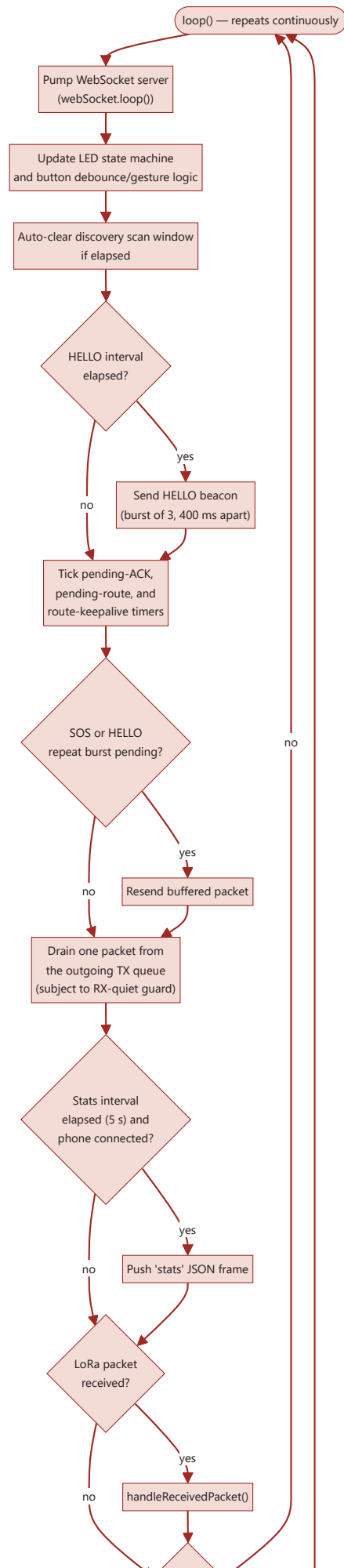
4.1 Hardware Platform and Radio Configuration

Each Ranger node is built on a "Rola" ESP32 board variant with a discrete SX127x-family LoRa transceiver connected over SPI, plus three status LEDs (green / yellow / red) and two push buttons — a dedicated SOS button and a multi-function button supporting single-, double-, and long-press gestures. The radio is configured once at boot for a deliberately balanced range/throughput profile:

Parameter	Value	Rationale
Frequency	433 MHz	License-friendly ISM band with good foliage/obstacle penetration for the target deployment regions
Spreading Factor	SF9	Balances range against airtime — SF7 trades range for speed, SF12 trades speed (and battery) for maximum range; SF9 sits in the middle for a mesh expecting a handful of hops rather than one very long link
Bandwidth	125 kHz	Standard LoRaWAN-compatible bandwidth; narrower bandwidths trade throughput for marginally better sensitivity
Coding Rate	4/5	Lightest forward-error-correction overhead LoRa offers, favouring throughput since the mesh already retries at a higher layer
TX Power	20 dBm	Maximum permitted output via the PA_BOOST pin, maximising single-hop range to minimise the number of relays a message needs
Sync Word	0xF3	Private network isolation — rejects packets from any co-located LoRa network using the public default sync word
CRC	Enabled	Hardware-level corruption detection before a packet is ever handed to the mesh layer

4.2 Program Structure

The firmware is a single translation unit running the classic Arduino `setup()` / `loop()` pattern. `setup()` brings up serial logging, configures I/O pins, starts the Wi-Fi access point and WebSocket server, initialises the LoRa radio (halting permanently with a red LED if the radio fails to respond — a deliberate fail-hard rather than a silent degraded mode, since a node with no radio has no purpose), and applies the radio parameters from Section 4.1. Every subsequent iteration of `loop()` is non-blocking and poll-driven:



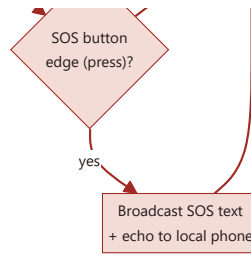


Figure 4.1 — One iteration of the firmware's main loop. Every stage is a bounded, non-blocking check; nothing in this path ever calls `delay()`, so the loop can execute many thousands of times per second and remain responsive to both radio traffic and the local phone.

4.3 Concurrency Model — Design Decision

DESIGN DECISION

Chosen: a single-threaded, poll-based main loop with no FreeRTOS tasks, no hardware timers, and no interrupt service routines — every event (an incoming LoRa packet, a button press, a WebSocket frame) is detected by checking a condition once per loop iteration rather than by being pushed asynchronously.

Alternatives considered: the ESP32's dual-core Xtensa architecture and FreeRTOS base would comfortably support splitting radio I/O, mesh routing, and the WebSocket server across separate tasks or even cores, and using `attachInterrupt()` for the SOS button would give sub-millisecond response latency instead of a loop-iteration's worth of jitter.

Why the simpler model won: the mesh routing state (route table, neighbor table, pending-ACK table, duplicate cache) is shared, mutable, and accessed from what would otherwise be multiple concurrent contexts — a multi-task design would need locking or careful message-passing around every one of those structures, on a microcontroller where a subtle race condition is exceptionally hard to reproduce and debug in the field. A single-threaded loop makes every state mutation atomic by construction: nothing can observe a table mid-update. Given that the loop body has no blocking calls, the practical latency cost is a few hundred microseconds per iteration at worst — immaterial next to LoRa's own multi-hundred-millisecond airtime per packet — while the correctness benefit (no possible race between, say, a route-table read during forwarding and a route-table write during RREP processing) is substantial. The one place this shows up as a real trade-off is button latency (bounded by loop iteration time rather than true interrupt latency), which the design accepts as invisible to a human pressing a physical button.

Mesh Networking and Routing Protocol

This section describes the core intellectual contribution of the firmware: a compact, reactive mesh-routing protocol purpose-built for a small number of intermittently-in-range LoRa nodes, fitted into the memory and airtime budget of an ESP32. It borrows the reactive philosophy of AODV (Ad hoc On-Demand Distance Vector routing) — routes are discovered only when needed, not maintained proactively for every possible destination — while simplifying several AODV mechanisms (sequence numbers, precursor lists) that add robustness at a cost the target deployment does not need to pay.

5.1 Packet Format

Every packet on the mesh begins with a fixed 31-byte header, hand-packed field by field (rather than cast from a C struct) specifically to avoid compiler-inserted padding — a struct-over-wire approach would waste several bytes per packet to alignment, which matters on a link where every byte costs airtime.

Offset	Field	Size	Meaning
0	<code>type</code>	1 byte	Packet type (Table 5.1)
1–2	<code>msgId</code>	2 bytes	Originator-assigned message ID, big-endian; the key (with <code>src</code>) used for de-duplication and ACK matching
3–10	<code>src</code>	8 bytes	Original source node ID, zero-padded
11–18	<code>dst</code>	8 bytes	Destination node ID, or "*" for broadcast
19–26	<code>prevHop</code>	8 bytes	The node that physically transmitted <i>this specific copy</i> — distinct from <code>src</code> , and re-stamped by every relay; the basis for neighbor discovery and reverse-route learning
27	<code>ttl</code>	1 byte	Hops remaining before the packet is silently dropped
28	<code>hop</code>	1 byte	Hops travelled so far (diagnostic / used to build reverse routes)
29	<code>priority</code>	1 byte	0 = normal chat, 1 = mesh control traffic, 2 = SOS (always wins queue arbitration)
30	<code>payloadLen</code>	1 byte	Length of the payload that follows (0–180 bytes)

Header size 31 bytes; maximum payload 180 bytes; maximum packet size 211 bytes.

5.2 Packet Types

Type	Value	Purpose
<code>PKT_DATA</code>	1	An actual application-layer message (chat text, a location sentinel, a pairing handshake, "BEEP") — unicast or broadcast
<code>PKT_ACK</code>	2	"I, the destination, received message <code>msgId</code> "
<code>PKT_HELLO</code>	3	A one-hop-only beacon: "I am node <code>src</code> " (TTL fixed at 1, never forwarded)
<code>PKT_RREQ</code>	4	Route request — "does anyone know how to reach <code>dst</code> ?" (floods outward)
<code>PKT_RREP</code>	5	Route reply — "I am (or know how to reach) <code>dst</code> ; here is the path back" (unicast, retraces the reverse route)
<code>PKT_DISCOVER</code>	6	App-triggered "who is out there?" scan (floods outward)
<code>PKT_DISCOVER_REPLY</code>	7	A node's answer to a discovery scan, unicast back toward the requester
<code>PKT_SOS_ACK</code>	8	Lightweight, best-effort, per-node confirmation of an SOS broadcast (no retry)
<code>PKT_NACK</code>	9	A relay's "I have no route to your destination" failure signal, sent back toward the original sender

5.3 Neighbor Discovery — HELLO Beacons

Every node broadcasts a `PKT_HELLO` beacon every 15 seconds (as a burst of three transmissions, 400 ms apart, for resilience against a single lost packet), always with TTL fixed at 1 so it is never relayed — a HELLO's entire purpose is to prove *direct* reachability. However, HELLO beacons are not the only source of neighbor knowledge: because every received packet's header carries `prevHop` — the node that physically transmitted this exact copy — the firmware opportunistically learns "I can hear this neighbor directly" from *any* traffic, including ordinary chat forwarding, well before the next scheduled HELLO would have told it the same thing.

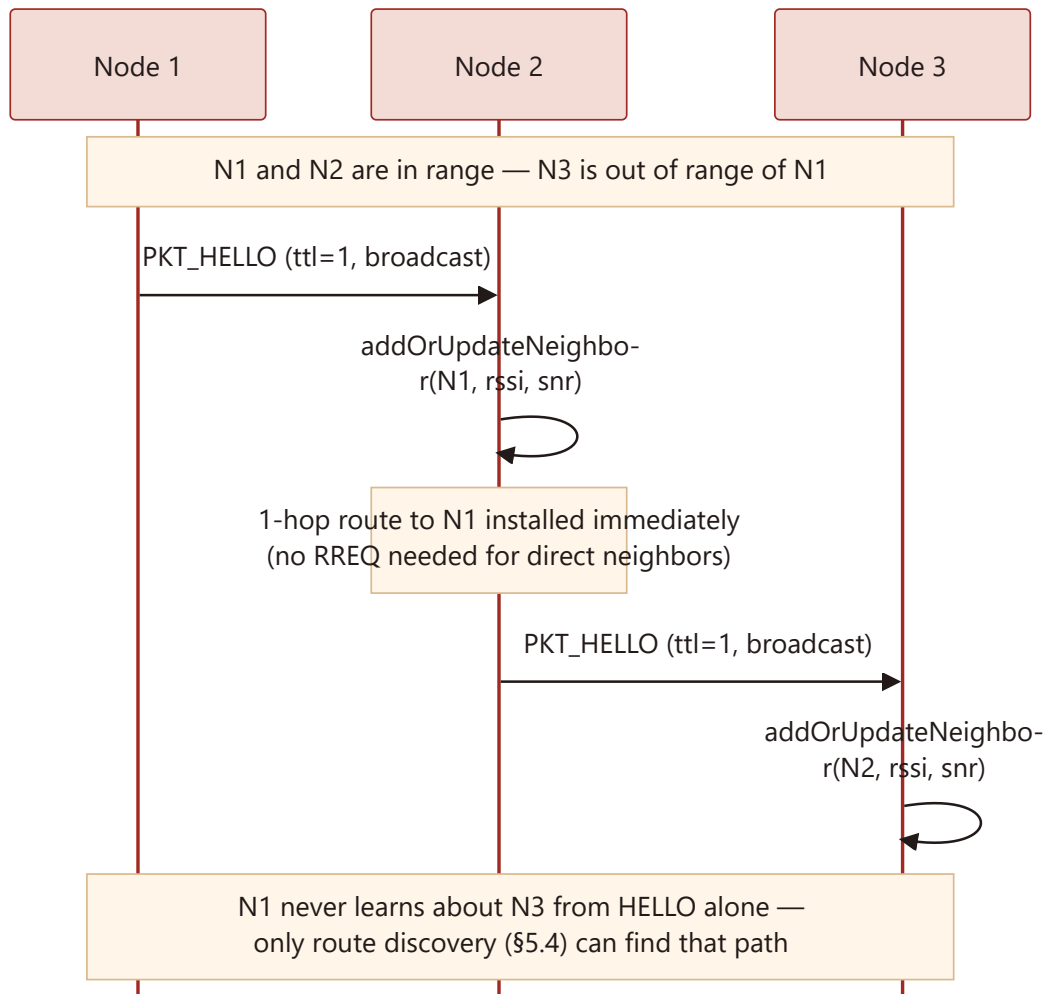


Figure 5.1 — HELLO beacon exchange. HELLO establishes only *direct* (one-hop) neighbor relationships; multi-hop reachability is never inferred from HELLO traffic and must go through explicit route discovery.

5.4 Reactive Routing — Route Request / Route Reply

When a node needs to send a unicast packet to a destination it has no known route for, it defers the send and floods a **PKT_RREQ** (route request) addressed to that destination, with TTL capped at **MESH_TTL_MAX = 6** hops. Every node that relays the RREQ — including the destination itself — learns a *reverse* route back to the original requester for free, because the RREQ's header already carries the requester's ID and the immediate relay's **prevHop**. When the RREQ finally reaches its target, that node replies with a unicast **PKT_RREP**, which retraces the reverse path exactly, installing a

forward route at every hop along the way. Figure 5.2 shows this for the three-node topology from Section 3.1.

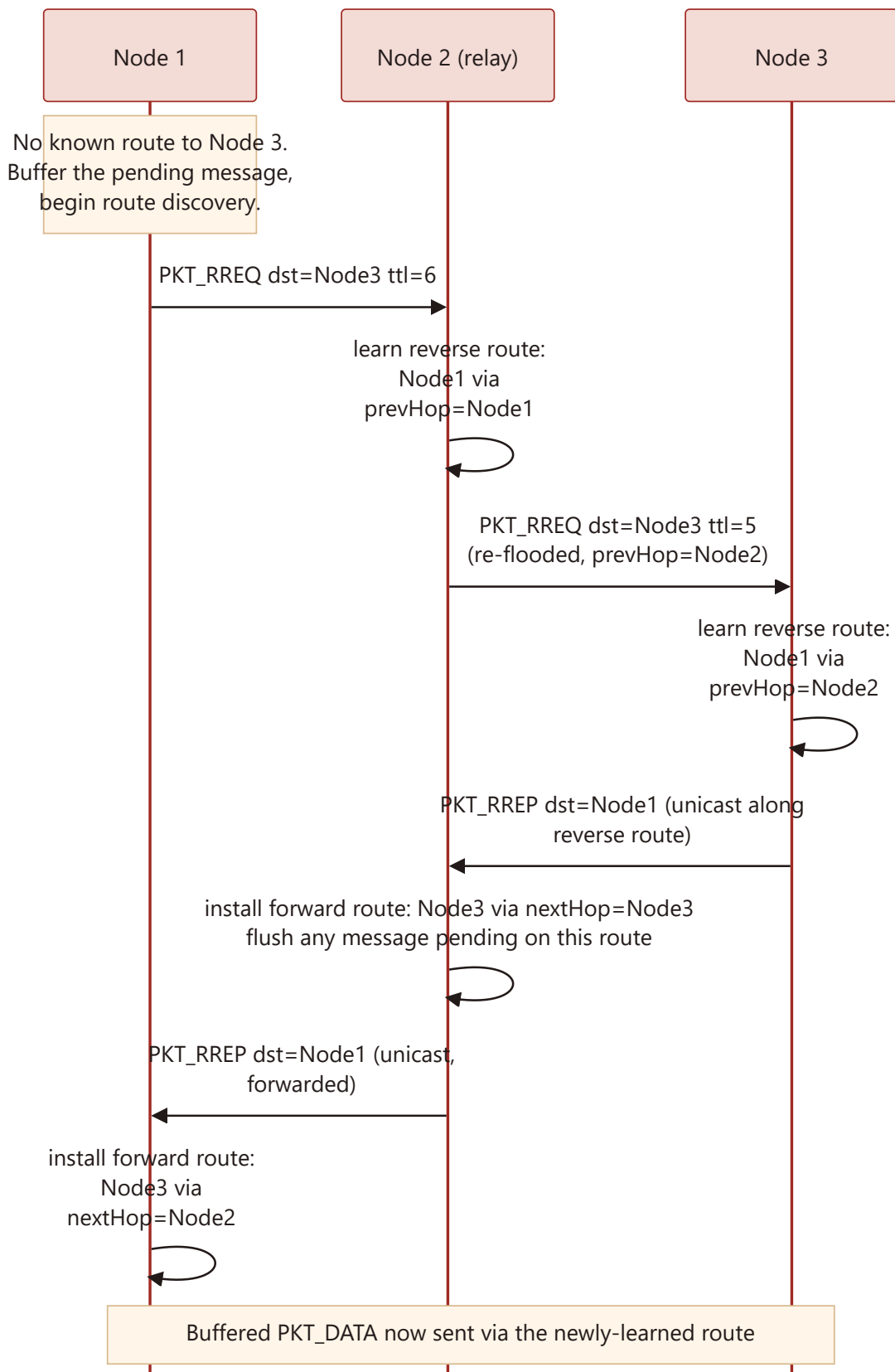


Figure 5.2 — Reactive route discovery (RREQ / RREP). A single discovery round installs routes at every intermediate hop in both directions, not just at the endpoints — a subsequent message from Node 2 to either Node 1 or Node 3 needs no discovery of its own.

Route discovery is bounded, not open-ended: at most `MAX_RREQ_ATTEMPTS = 2` attempts (one initial send and one retry after `RREQ_RETRY_INTERVAL_MS = 3000` ms with no reply), inside an overall `ROUTE_DISCOVERY_TIMEOUT_MS = 8000` ms budget, after which the pending message is reported to the phone as a delivery failure rather than retried indefinitely. Routes themselves expire after `ROUTE_EXPIRY_MS = 60000` ms of disuse, but a *proactive keepalive* mechanism refreshes a multi-hop route that is still being actively used and is within `ROUTE_KEEPA_LIVE_MARGIN_MS = 15000` ms of expiring, firing one more RREQ before the route would otherwise lapse — this specifically targets multi-hop routes (`hopCount > 1`), since a direct one-hop neighbor route is re-established for free by ordinary HELLO/opportunistic learning and never needs a costly flood to refresh.

5.5 Forwarding and Flooding

Every packet not addressed to the receiving node passes through one of two forwarding primitives, selected by the packet's semantics: **flood forwarding** (for RREQ, DISCOVER, and broadcast DATA/SOS — the packet is re-broadcast to everyone) and **unicast forwarding** (for RREP, ACK, NACK, and unicast DATA — the packet is sent only toward the specific next hop found in the route table). Both share a common trunk: decrement TTL, increment the hop counter, re-stamp `prevHop` to this node's own ID, and enqueue for transmission. A packet whose TTL has already reached zero is silently dropped rather than forwarded, bounding how far any single packet — and any routing loop, however unlikely — can travel.

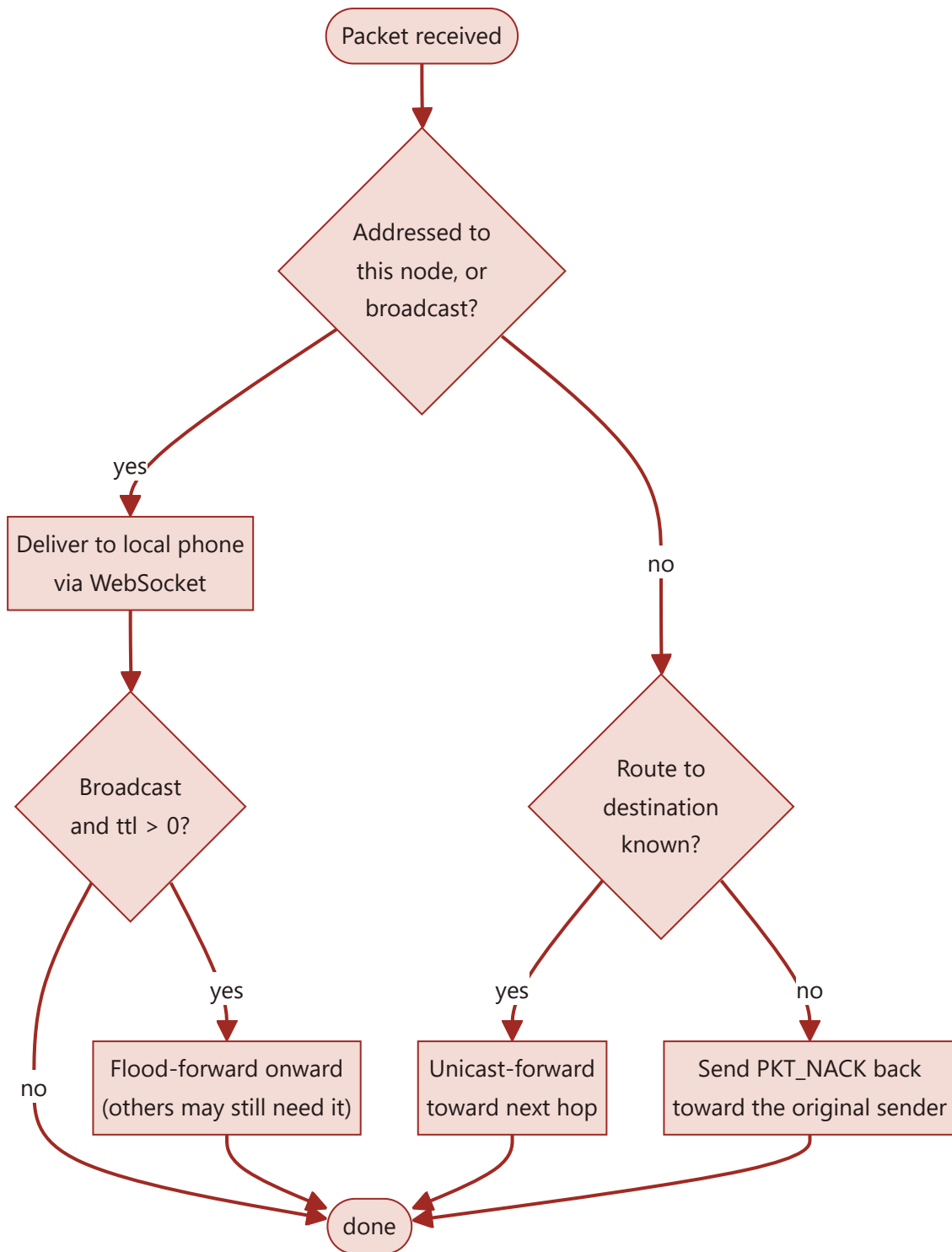


Figure 5.3 — Forwarding decision for a received unicast/broadcast packet. A relay that cannot forward a unicast packet does not simply drop it silently — it actively signals the failure back toward the sender (Section 5.7), so the reliability layer can react promptly instead of waiting for a full ACK timeout.

5.6 Duplicate Suppression

Because both flooding and retransmission can deliver the same logical packet to a node more than once, every node keeps a rolling cache of (originalSource, msgId) pairs it has already processed (MAX_SEEN = 40 entries, each expiring after SEEN_CACHE_EXPIRY_MS = 30000 ms, evicted least-recently-used when full). A packet matching an entry already in the cache is dropped before any type-specific handling runs — with one deliberate exception: a duplicate unicast PKT_DATA addressed to this node still triggers a fresh PKT_ACK reply (without re-delivering the text to the phone a second time), since the most likely explanation for seeing the same message twice is that the *original* ACK, not the message, was the packet that got lost.

5.7 Reliability — Acknowledgement, Retry, and NACK

Every unicast PKT_DATA send is tracked in a pending-acknowledgement table (MAX_PENDING_ACK = 10 slots) with a per-message retry budget. If no PKT_ACK arrives within ACK_TIMEOUT_MS = 5000 ms, the exact same raw bytes (same msgId) are retransmitted, up to MAX_SEND_RETRIES = 5 times. Exhausting all direct retries does not immediately report failure to the phone — it first invalidates the (evidently stale) route and attempts *one* full route-rediscovery-and-retry cycle; only if that second cycle also fails does the firmware report the delivery as permanently failed. A mid-path relay that discovers it has no route for a packet does not wait for the sender's ACK timer to expire at all — it immediately emits a PKT_NACK back toward the original sender, which triggers the same route-invalidation and one-time-recovery logic early, shortening the failure-detection latency considerably compared to relying on the timeout alone.

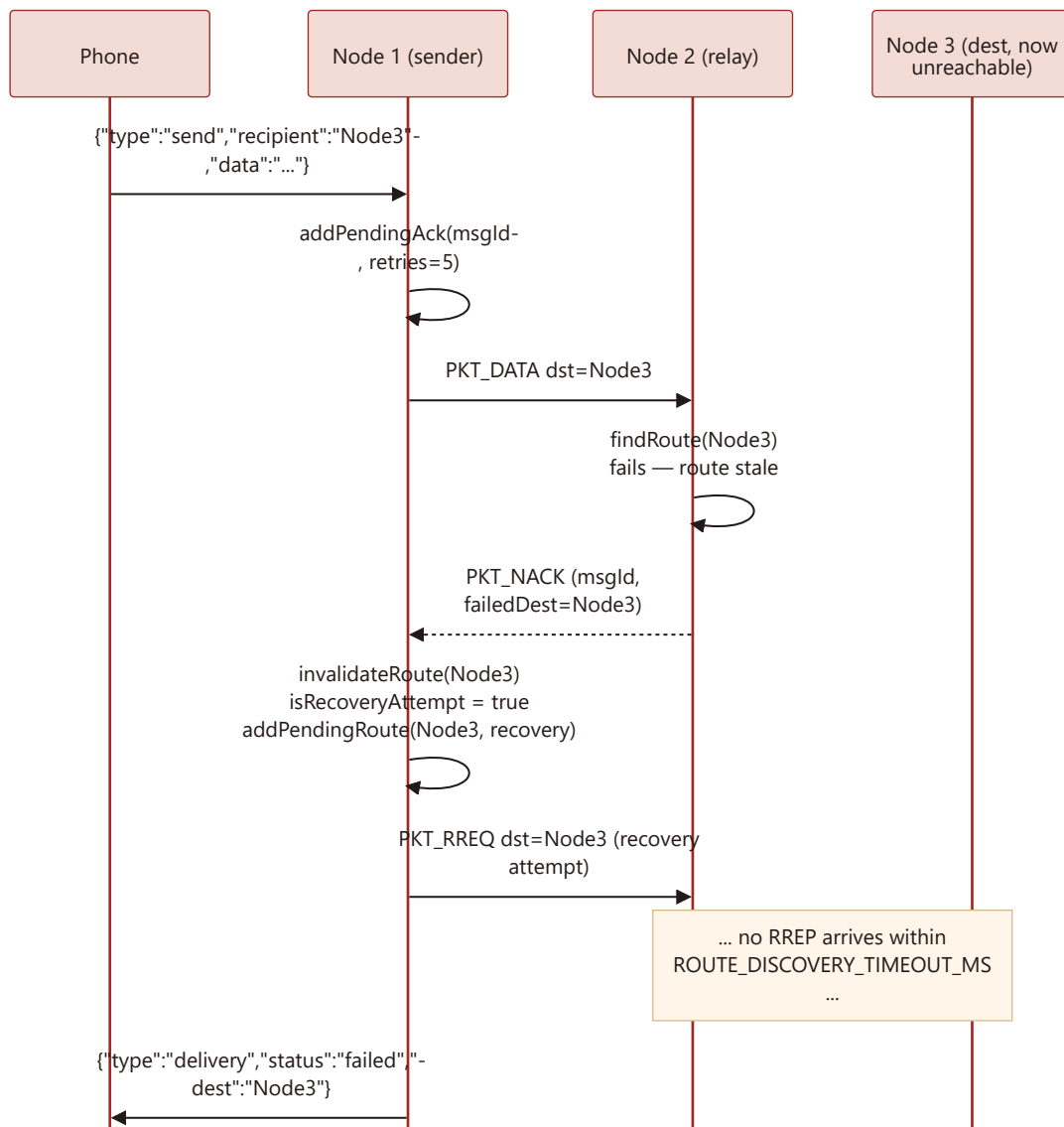


Figure 5.4 — NACK-accelerated failure detection and bounded recovery. The relay's NACK lets the sender begin route recovery immediately instead of waiting out a full ACK timeout; the recovery itself is capped at exactly one extra cycle so a genuinely unreachable node cannot cause an unbounded retry loop.

Broadcast and SOS traffic use a different reliability strategy entirely: acknowledging every broadcast individually to every receiver would create an "ACK storm" that scales with the number of nodes in range. Instead, a broadcast is sent as a burst of `SOS_TOTAL_SENDS = 3` copies (250 ms apart, with jitter) relying on the duplicate cache at each receiver to absorb the repeats, and each receiving node sends back one best-effort, non-retried `PKT_SOS_ACK` — enough for the sender to build mesh-wide delivery confidence (Section 11) without the cost of guaranteed per-recipient delivery.

5.8 Outgoing Queue and Priority Arbitration

All transmissions — original sends and forwards alike — pass through a single 14-slot outgoing queue rather than keying the radio immediately, so that a burst of forwarding work never blocks the local phone's own traffic. Each entry carries the packet's priority (normal / control / SOS); when the queue is full, an incoming higher-priority packet preempts the lowest-priority queued entry rather than being dropped outright, guaranteeing that an SOS broadcast is never silently discarded because the queue happened to be full of ordinary chat traffic. The queue is drained at most one packet per loop iteration, gated by a short `RX_QUIET_GUARD_MS = 25` ms hold-off after any received packet — a lightweight, poor-man's collision-avoidance measure, since LoRa (unlike Wi-Fi) has no hardware carrier-sense/collision-detection of its own.

5.9 Routing Design Alternatives Considered

DESIGN DECISION — REACTIVE VS. PROACTIVE ROUTING

Alternative considered: a proactive, link-state or distance-vector protocol (in the style of OLSR or classic RIP) in which every node periodically floods its full routing table so that a route to any destination is always immediately available, with no on-demand discovery delay.

Why reactive (AODV-style) was chosen instead: in this deployment's expected scale — a handful to a few dozen nodes, many of which may be powered down or out of range for long stretches — the airtime cost of continuously flooding full routing tables would dwarf the airtime actually spent on user traffic, on a link where every byte of airtime is scarce and shared. A reactive protocol pays a one-time latency cost (route discovery, typically well under a second on a small mesh) only for destinations that are actually being talked to, and pays nothing at all for the majority of node pairs that never communicate directly. The opportunistic route-learning described in Sections 5.3 and 5.4 — where ordinary traffic and even overheard RREQ/RREP flooding silently populate routes as a side effect — recovers much of proactive routing's "route is already there" convenience without its continuous flooding cost.

DESIGN DECISION — SIMPLIFIED AODV, NOT FULL AODV

Full AODV specifies per-destination sequence numbers to guarantee loop-freedom and route freshness even under adversarial reordering, and precursor lists so a broken link can trigger a targeted route error to exactly the nodes using it. ATCS's routing omits both: loop safety is instead guaranteed structurally by the strict TTL decrement on every hop (a packet can physically never travel more than `MESH_TTL_MAX` hops, so even a transient loop self-terminates quickly), and a broken route is discovered reactively (by NACK or ACK timeout, Section 5.7) rather than proactively announced. This trades a theoretically small amount of extra latency on route breakage for a substantially smaller and simpler implementation — an appropriate trade for a network of this scale, where the added complexity of full AODV's freshness guarantees would buy correctness margin the deployment does not need.

Communication Protocol

— Phone ↔ Firmware

6.1 Transport Choice — WebSocket over Wi-Fi

DESIGN DECISION

Chosen: the ESP32 runs a persistent WebSocket server (port 8765) that the phone connects to once and keeps open for the life of the session, exchanging small JSON frames in both directions.

Alternatives considered: a REST-style HTTP polling API (the phone repeatedly asks "anything new?"), or a raw TCP/serial byte protocol with a custom framing format.

Why WebSocket won: HTTP polling would force a trade-off between battery/airtime waste (frequent polling) and latency (infrequent polling) for a use case — incoming chat messages, location updates, delivery confirmations — that is fundamentally push-driven from the firmware's side. A raw byte protocol would avoid JSON's parsing overhead but would require the app to hand-roll framing, and JSON's self-describing, forward-compatible structure proved valuable in practice: several fields (`hops` , `via` , sequence numbers) were added to the protocol over the project's life without breaking older frames, since a receiver simply ignores JSON keys it does not recognise. `ArduinoJson` 's memory footprint (a single reused 1024-byte document) is modest enough for the ESP32's RAM budget that this convenience carries negligible cost.

6.2 Firmware → Phone Messages

type

Key fields

device_info

deviceId, deviceName, frequency, spreadingFactor, bandwidth

message

sender, data, rssi, broadcast?, hops?, via?

delivery

status: delivered\|failed\|sos_received, dest?, from?

neighbor

deviceId, rssi, snr

discovery

deviceId, rssi, hops?, via?

discover /
reconnect

—

pong

—

type	Key fields
stats	pktSent, appMsgSent, messagesReceived, uptime, connectedClients, pktForward

6.3 Phone → Firmware Messages

type	Fields	Firmware action
send	recipient, data	Encode and queue a unicast or broadcast <code>PKT_DATA</code>
beep	recipient	Shorthand for <code>send</code> with the literal payload <code>"BEEP"</code>
discover	—	Open a scan window and flood a <code>PKT_DISCOVER</code>
ping	—	Application-level liveness probe; firmware replies <code>pong</code> directly to the requesting client

6.4 Application-Layer Sentinels

The firmware treats a `PKT_DATA` payload as opaque bytes — it never inspects message content beyond the literal substring `"SOS"` for the emergency LED (Section 11). All of the richer behaviours below — location sharing and pairing — are therefore entirely an application-layer convention, implemented by prefixing plain text with a short sentinel that the receiving app recognises and decodes, while an older or unaware receiver would simply display the sentinel as literal (if odd) chat text rather than crash or misbehave.

Sentinel	Meaning
<code>##LREQ##</code>	Requester asks the recipient to share their GPS position
<code>##LRESP##seq,lat,lng,acc</code>	A GPS fix — used identically for a one-time reply and for every update in a live-share session; the leading sequence number lets the receiver discard a delayed retry of a stale fix that arrives after a fresher one already has
<code>##LSTOP##</code>	The responder explicitly ended a live-share session
<code>##PAIR_REQ##<name></code>	Node A requests to pair, announcing its display name
<code>##PAIR_ACK##<name></code>	Node B accepts the pairing, announcing its own display name

6.5 Liveness and Heartbeats

Connection liveness is checked at two independent layers, because a WebSocket reporting `readyState === OPEN` is not, on Android, proof that the link is actually alive — the OS can freeze the app's JavaScript timers while backgrounded, or drop Wi-Fi entirely, without ever firing a clean close event.

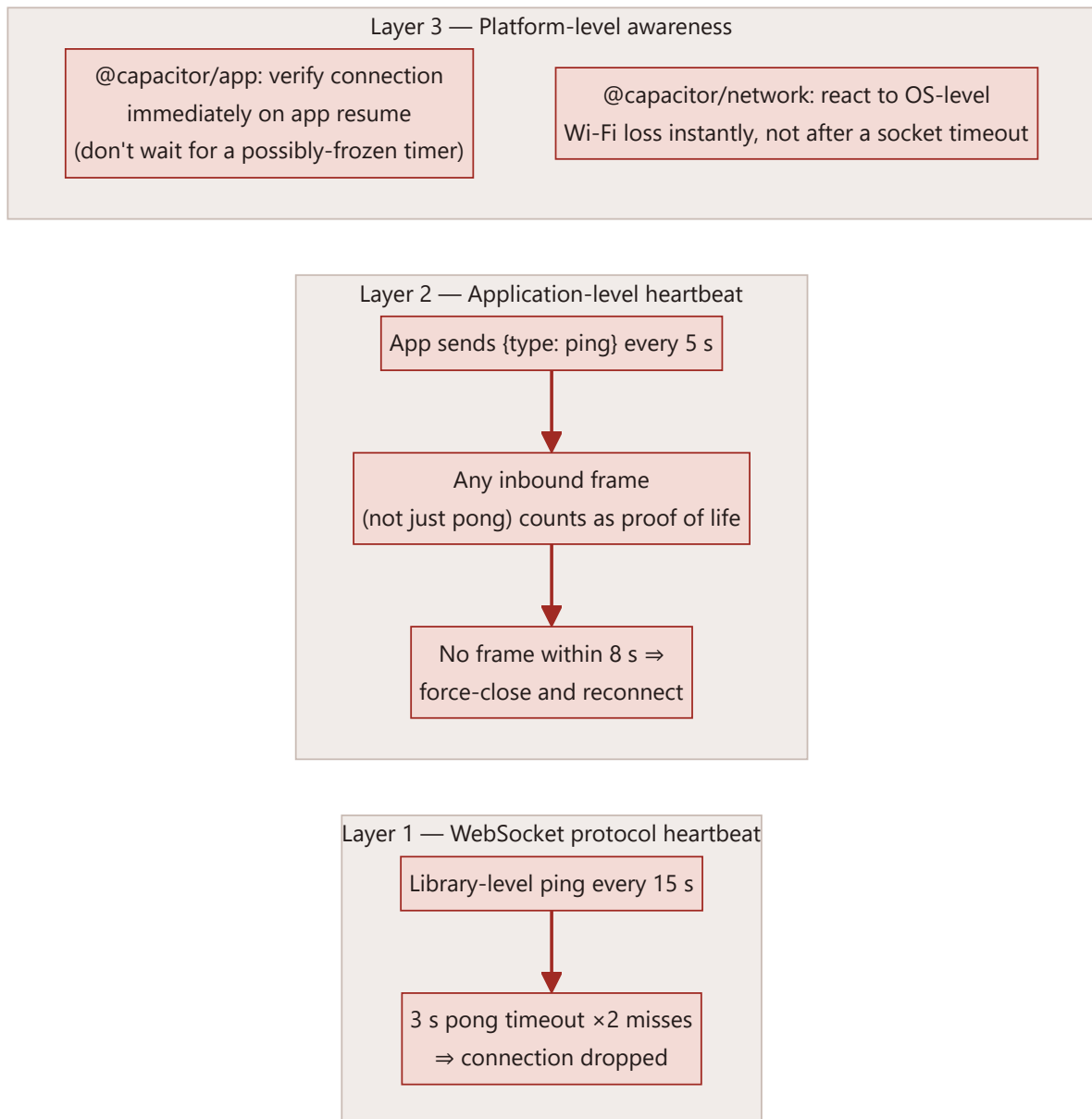


Figure 6.1 — Three independent liveness layers. Each catches a different failure mode the others would miss; together they close the gap between "the socket object still looks open" and "the link is actually usable right now."

6.6 Worked Example — A Message's Full Journey

Figure 6.2 traces one chat message across the entire stack, from the sender's phone through two relaying nodes to the recipient's phone, and the acknowledgement travelling back — tying together every layer described in Sections 5 and 6 into a single concrete scenario.

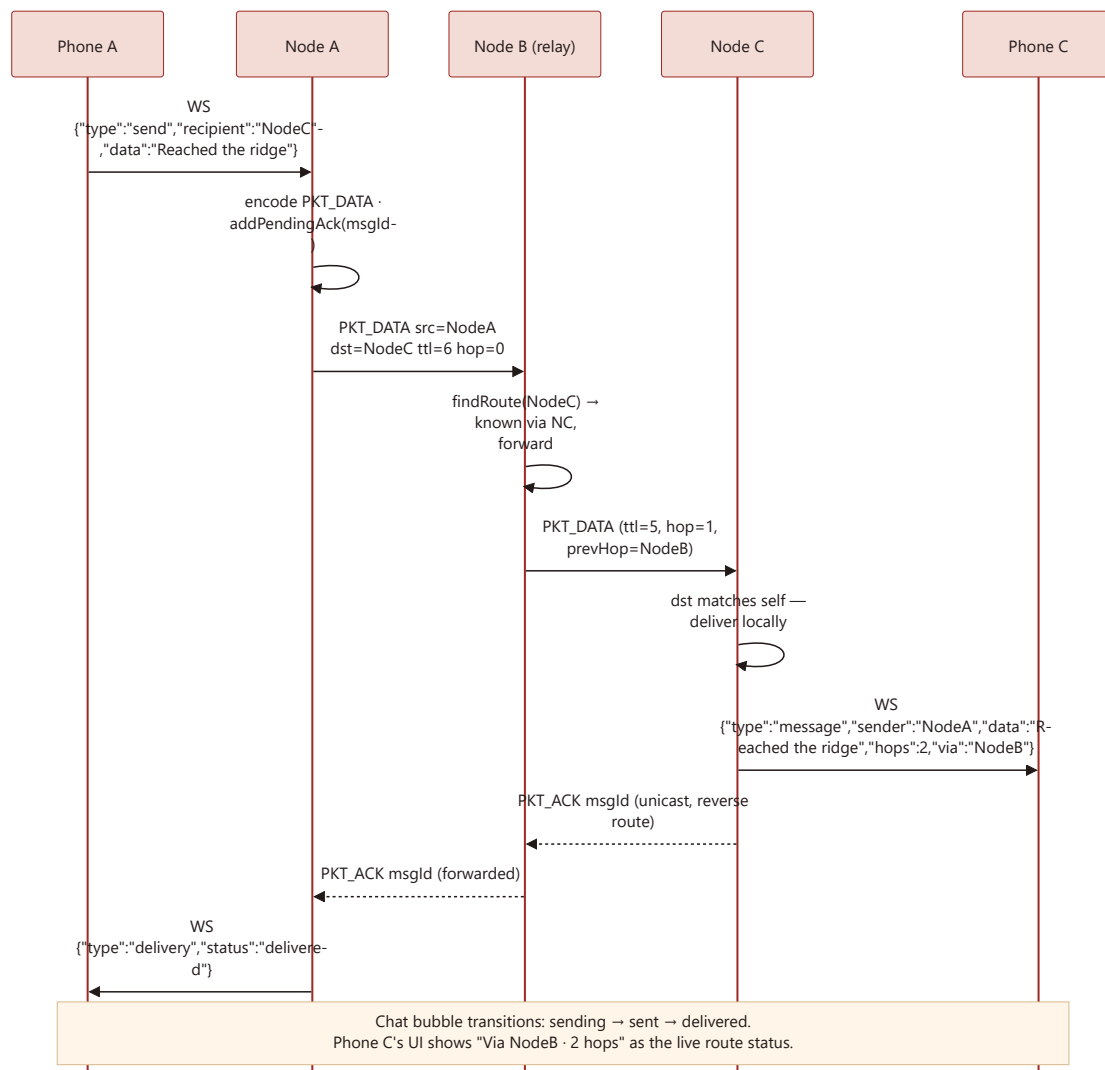


Figure 6.2 — End-to-end journey of a single relayed chat message. Every hop re-stamps `prevHop` and decrements `ttl`; the acknowledgement retraces the exact reverse path rather than being independently routed, so it requires no additional route discovery.

Messaging Subsystem

7.1 Send Flow and Delivery State Machine

Sending a message is optimistic: the message is appended to the conversation and rendered immediately in a `sending` state before the firmware has confirmed anything, so the interface never blocks on radio latency. From there its status is driven entirely by asynchronous events arriving from the firmware, plus one client-side timer that exists purely to keep the user informed while the mesh is still working on it.

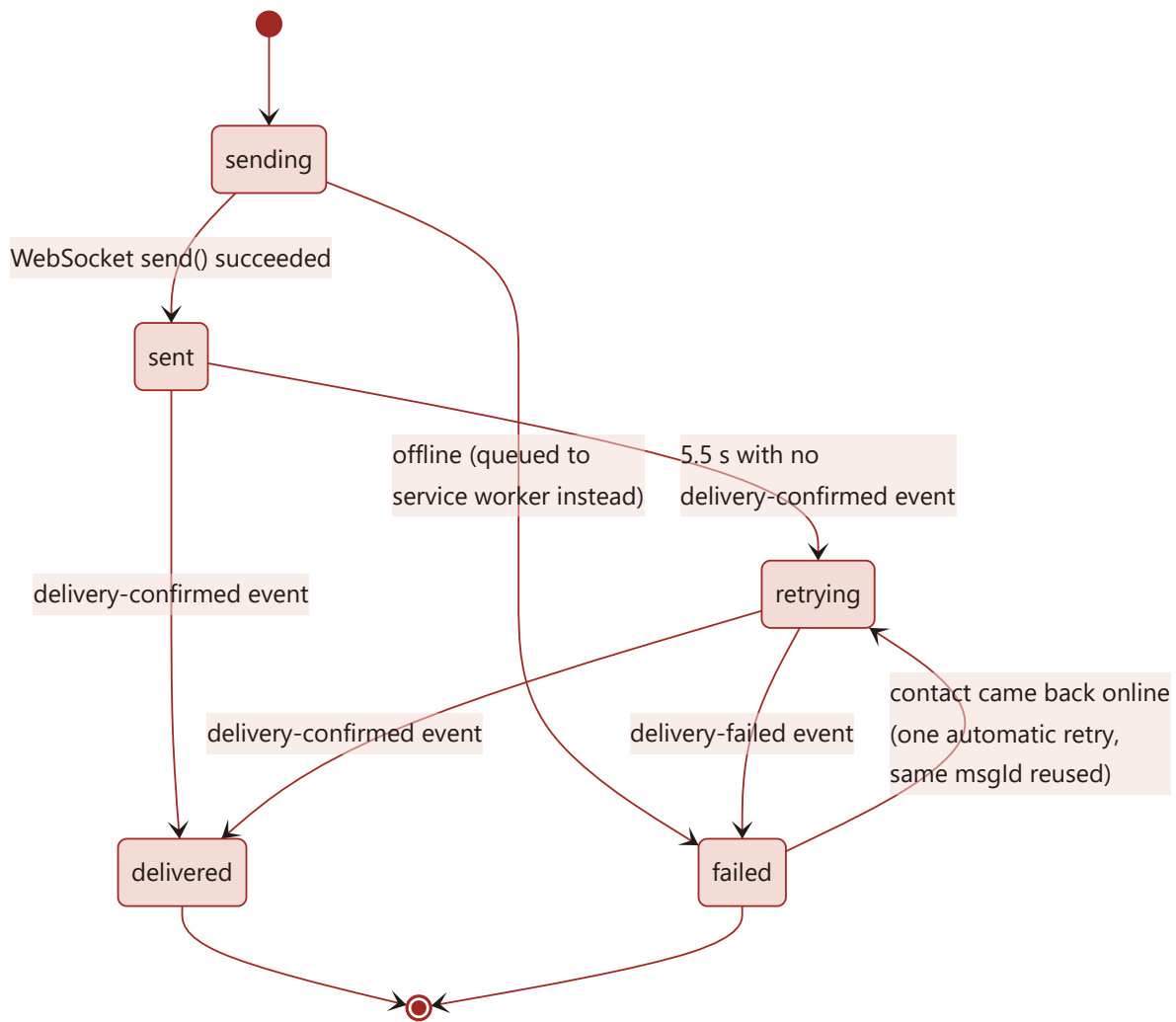


Figure 7.1 — Client-side message delivery state machine. The `retrying` visual state is a pure UI affordance — the firmware itself is already retrying transparently underneath (Section 5.7); this state exists so the user is not left staring at an ambiguous "sent" checkmark for several seconds while that happens. A permanently failed send is retried exactly once, automatically, the moment the recipient is next observed online — never in an unbounded loop.

Broadcast (Emergency channel) messages skip the per-recipient ACK/retry path entirely, since the mesh has no concept of "the one recipient" for a broadcast — their status instead reflects `sos_received` confirmations arriving from individual nodes (Section 11).

7.2 Receive Flow and Deduplication

An incoming `message` frame is filed into one of two places: the shared Emergency thread (if the underlying LoRa packet was addressed to `"*"`) or a private 1:1 thread keyed by sender ID. Because the firmware itself may deliver logically-duplicate broadcasts (Section 5.7's three-copy SOS burst), the application applies its own dedup window before rendering — a shorter one (1000 ms) for ordinary 1:1 traffic, where a fast legitimate repeat should still show, and a longer one (4000 ms) for broadcasts, matched to the burst's own resend spacing. A message from a sender not already in the contact list — the common case for an unsolicited broadcast — automatically registers that sender as a new contact, which is what lets a recipient tap through into a private conversation with a stranger who happened to broadcast into range.

7.3 Offline Queueing

If a send is attempted while the phone's own WebSocket link to its node is down, the message is marked `failed` immediately and, if a service worker is active, also handed off via `postMessage` to be retried through the Background Sync API once connectivity returns — a deliberately separate mechanism from the mesh-level retry described in Section 5.7, which only ever applies to a message that was *successfully* handed to the firmware in the first place.

Reachability and Signal Quality Tracking

The application deliberately maintains two classifiers for a contact's link health, and — by explicit design rule — never lets one influence the other:

Classifier	Input	States	Never influenced by
Reachability	Time since last heard (any packet)	online (<22 s) / stale (<45 s) / offline (≥45 s)	Signal strength — a weak but present link is still "online"
Signal quality	Most recent RSSI reading	strong (≥−85 dBm) / good (≥−100 dBm) / weak / unknown	Elapsed time — a stale reading is shown muted with its age, never silently refreshed

DESIGN DECISION

Keeping these independent avoids a specific, easy-to-introduce class of bug: a node with a weak but still-arriving signal must never be shown as "offline" (it is reachable, just faint), and a node whose last-known signal happened to be excellent must never be shown as "online" purely because that old reading looked good, once too much time has passed since anything was actually heard from it. The 22 s / 45 s reachability thresholds are calibrated directly against the firmware's 15 s HELLO interval — 22 s tolerates one late beacon without a false "stale" flicker, while 45 s (three missed beacons) is unambiguous evidence the node is genuinely gone.

Because "silence" is not a discrete event a listener can subscribe to, a periodic monitor re-evaluates every contact's reachability every 5 seconds regardless of whether new traffic has arrived, and raises a transient banner the moment a contact's classification actually changes state — including triggering the one automatic message retry described in Figure 7.1 the instant a contact transitions back to **online**. Every inbound event that carries a signal sample also threads through the hop-distance and relaying-neighbor fields from the mesh header (Section 5.1), which is what allows the interface to show a live "Direct connection" or "Via Node2 · 2 hops" status per contact, rather than a one-off snapshot from the last discovery scan.

Navigation and Compass Subsystem

9.1 From Offline Maps to Compass-Only — A Pivot

CHALLENGE AND MITIGATION

An earlier iteration of ATCS included a full offline-map navigation mode: Leaflet.js rendering a locally-bundled tile set, with a one-time download-on-first-internet-access system so the map would keep working forever afterward with zero connectivity. This proved to be a substantial engineering surface for the value it delivered: sourcing and packaging map tiles for an arbitrary deployment region is inherently a connectivity-dependent bootstrapping problem in a system whose entire premise is that connectivity cannot be assumed, and the download pipeline itself hit a genuine external infrastructure failure during development (an unexplained propagation failure serving files from `raw.githubusercontent.com`, which was ultimately worked around by switching to GitHub Releases as the distribution mechanism instead).

Resolution: under project time constraints, the decision was made to remove the map subsystem entirely — the Leaflet dependency, the tile-download pipeline, and the Map UI — and rely exclusively on compass bearing-and-distance guidance toward a peer's last known GPS coordinates. This is a smaller, more honest feature: it does not attempt to show terrain or paths, only "how far, and in which direction," which is both simpler to make reliable and arguably more appropriate for guiding a person toward another *person* rather than toward a fixed map location. This is a representative example of a recurring theme in the project: several subsystems were built, tested, and then deliberately simplified once their actual cost-to-value ratio became clear under field conditions.

9.2 Bearing and Distance Mathematics

Both calculations are standard great-circle formulas operating on the WGS-84 spherical approximation (Earth radius 6,371,000 m), implemented once in `lib/geo.ts` and shared by every part of the app that needs them:

- **Distance** — the Haversine formula, accurate to well within GPS's own error margin at the ranges this system operates over.
- **Initial bearing** — the forward-azimuth formula (`atan2(sin Δλ · cos φ2, cos φ1 · sin φ2 - sin φ1 · cos φ2 · cos Δλ)`), giving the compass heading from the user's current position toward the target's last known position.

The needle angle actually rendered on screen is the difference between this GPS-derived bearing and the phone's own live compass heading (Section 9.3) — as the user physically turns, the needle rotates to compensate, always pointing at the target regardless of which way the phone itself is facing.

9.3 Sensor Fusion and Calibration

CHALLENGE AND MITIGATION

The most persistent navigation defect during field testing was a compass that pointed in a plausible-looking but simply wrong direction. The root cause was platform-specific: Android's plain `deviceorientation` event reports `alpha` relative to an arbitrary reference angle fixed the moment the sensor first activates — not relative to magnetic north — while iOS's `webkitCompassHeading` is always true-north-calibrated. The correct Android event, `deviceorientationabsolute`, fires only once the OS has fused magnetometer, gyroscope, and accelerometer data into a genuine magnetic-north heading — but on many devices both listeners fire for every physical sensor tick, and an earlier version of the code fed *both* into the same smoothing filter, meaning every other reading was, in effect, measured from a different, unrelated zero-point. The fix was twofold: prefer `deviceorientationabsolute` explicitly wherever it is available, and — critically — once even one genuine absolute event has been observed, silently discard any further relative-only events for the rest of the session rather than blending them in. A secondary calibration-quality heuristic inspects the last ten headings for either suspicious flatness (a stuck sensor) or implausibly large single-step jumps, surfacing a "move your phone in a figure-8" prompt only when the evidence actually warrants it.

Platform	Event	Heading reference
iOS	<code>deviceorientation</code> → <code>webkitCompassHeading</code>	Always true magnetic north; used as-is
Android (modern)	<code>deviceorientationabsolute</code>	Magnetic north, <code>heading = (360 - alpha) mod 360</code>
Android (fallback, older devices)	<code>deviceorientation</code>	Arbitrary reference — only trusted until a genuine absolute event has been seen

9.4 Smoothing Pipeline

GPS and compass sensor readings are noisy at the sample-to-sample level, so several independent exponential-moving-average (EMA) filters smooth the values actually rendered on screen. Each stage was tuned individually, and their interaction turned out to matter as much as any single stage in isolation.

CHALLENGE AND MITIGATION — COMPOUNDING SMOOTHING LAG

Distance-to-target passes through *two* smoothing stages by construction: the raw GPS fix is itself smoothed adaptively by the geolocation hook (more aggressively when accuracy is poor), and the compass screen then smooths its own derived distance value again on top of that. At the original tuning (a 0.30 blend factor for the second stage), these two filters compounded into visible lag while actually walking — the displayed distance measurably trailed the true distance by several metres, which read to a field tester as "the calibration is inaccurate" even though the underlying GPS fix was fine. Raising the second stage's blend factor to 0.55 roughly halved that lag while still damping genuine sample-to-sample jitter, without touching the first (adaptive, accuracy-aware) stage at all — the fix targeted the specific interaction between the two layers rather than either layer in isolation.

Smoothed quantity	Method	Purpose
Raw GPS position	Accuracy-adaptive EMA (no smoothing below 10 m error, up to 60% weight on the previous fix above 50 m error)	Damp GPS jitter without ever fabricating a placeholder accuracy value
GPS-derived bearing to target	Fixed-alpha EMA (0.30) with correct $\pm 180^\circ$ wraparound handling	Prevent the bearing from jumping erratically as successive fixes arrive
Compass needle angle	Fixed-alpha EMA (0.40)	Smooth sensor jitter without perceptible lag as the user turns
Displayed distance	Fixed-alpha EMA (0.55, raised from 0.30)	Damp jitter while minimising compounding lag with the position filter above
Displayed accuracy figures	Fixed-alpha EMA (0.25)	Stop the " $\pm N$ m" readout from visibly flickering every sample, without changing what the number means

A further refinement treats a GPS fix's accuracy as decaying with age rather than fixed at capture time: a fix that looked precise when it arrived becomes progressively less trustworthy for arrival detection the longer it sits unrefreshed, since the target may since have moved. The effective accuracy used for "have we arrived?" logic is inflated by an assumed walking pace (1.4 m/s) for every second since the fix was captured, capped so a very stale fix does not produce an absurd number — arrival itself is a simple, deterministic threshold (within 2 m of the smoothed distance) rather than a heuristic that tries to guess "close enough given GPS error," which an earlier version used and which could false-positive "arrived" at real separations of ten or more metres whenever GPS accuracy alone was merely poor.

9.5 Navigation Session Lifecycle

A navigation session is deliberately decoupled from whether the Compass screen is currently visible, so that dismissing it never has to mean abandoning it — a distinction that required correcting course once during development (an earlier iteration made the dismiss button jump straight to a different screen, which was reverted in favour of the minimize behaviour below after field feedback).

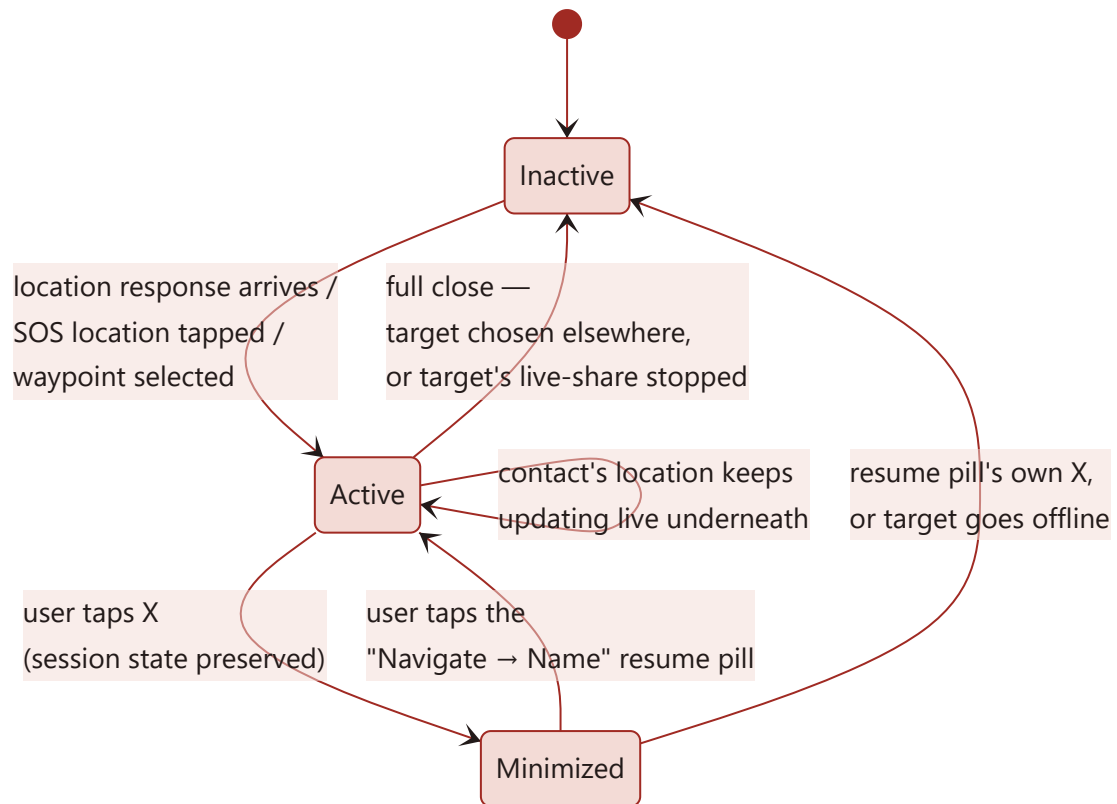


Figure 9.1 — Compass navigation session state. "Minimized" is not a paused state — GPS tracking, live-share reception, and reachability monitoring for the target all continue exactly as before; only the full-screen Compass UI itself is hidden, letting the user text or browse contacts while still being one tap away from resuming guidance.

Location Sharing

10.1 One-Time Request / Response

Location sharing always requires explicit consent from the person being located — there is no auto-accept path for an ordinary request. A requester's outgoing `##LREQ##` triggers a full-screen consent overlay on the responder's device; only "Share Live Location" begins transmission, and "Decline" ends the exchange with nothing sent. The moment the first fix arrives back at the requester, the pending request is resolved and the app transitions straight into an active Compass navigation session (Section 9.5) pointed at that position — a request is, from the requester's point of view, indistinguishable from "the compass just turned on."

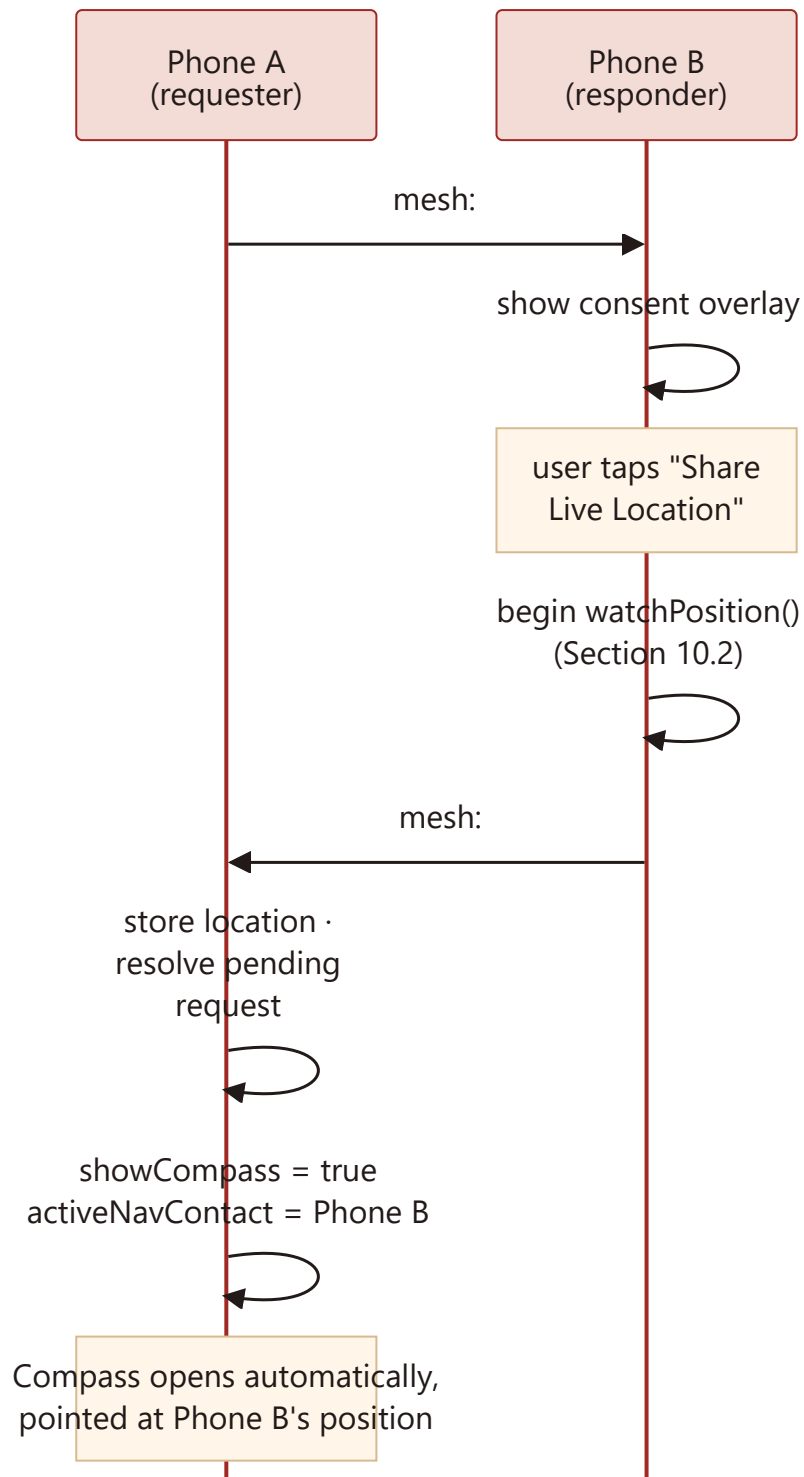


Figure 10.1 — One-time location request, resolving directly into navigation.

10.2 Continuous Live Sharing

Once consent is granted, the responder does not poll on a fixed timer — it opens a single continuous `watchPosition` GPS subscription and applies a send decision on every raw fix the OS delivers. An earlier design used a fixed-interval timer instead; that approach was found to interact poorly with Android's aggressive throttling of background GPS activity, and the constant is still present in the codebase as a historical remnant of that abandoned approach, superseded by the always-on watch below.

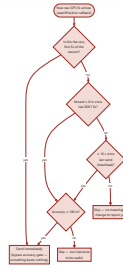


Figure 10.2 — Live-share send decision, evaluated on every GPS callback. Three conditions are OR'd together: always send the first fix, send on meaningful movement (above typical consumer-GPS jitter), or send on a heartbeat interval even while stationary so the requester has continuing proof the session is still alive.

A live-share session survives a temporary loss of the local Wi-Fi link by pausing rather than terminating — sends are simply skipped while offline and resume automatically the moment connectivity returns, with no need to re-request consent. Because the mesh transport retries failed unicast sends end-to-end but has no ordering guarantee across independently-sent messages, every fix carries a monotonically increasing per-device sequence number; a receiver drops any incoming fix whose sequence number is not newer than the last one it accepted, so a delayed retry of an older position can never visually snap the target backward on screen after a fresher position has already arrived.

Emergency (SOS) Subsystem

An SOS is, at the transport level, nothing more than a broadcast text message containing the substring **"SOS"** — triggered either by the dedicated hardware button on a Ranger node or by a user typing into the app's Emergency thread — but it receives distinct treatment at every layer above that: highest queue priority end-to-end (Section 5.8), a redundant three-copy burst rather than a single transmission (Section 5.7), an automatic best-effort location broadcast from the triggering device, and a persistent, non-auto-dismissing confidence banner that accumulates confirmations from every node that actually received it.

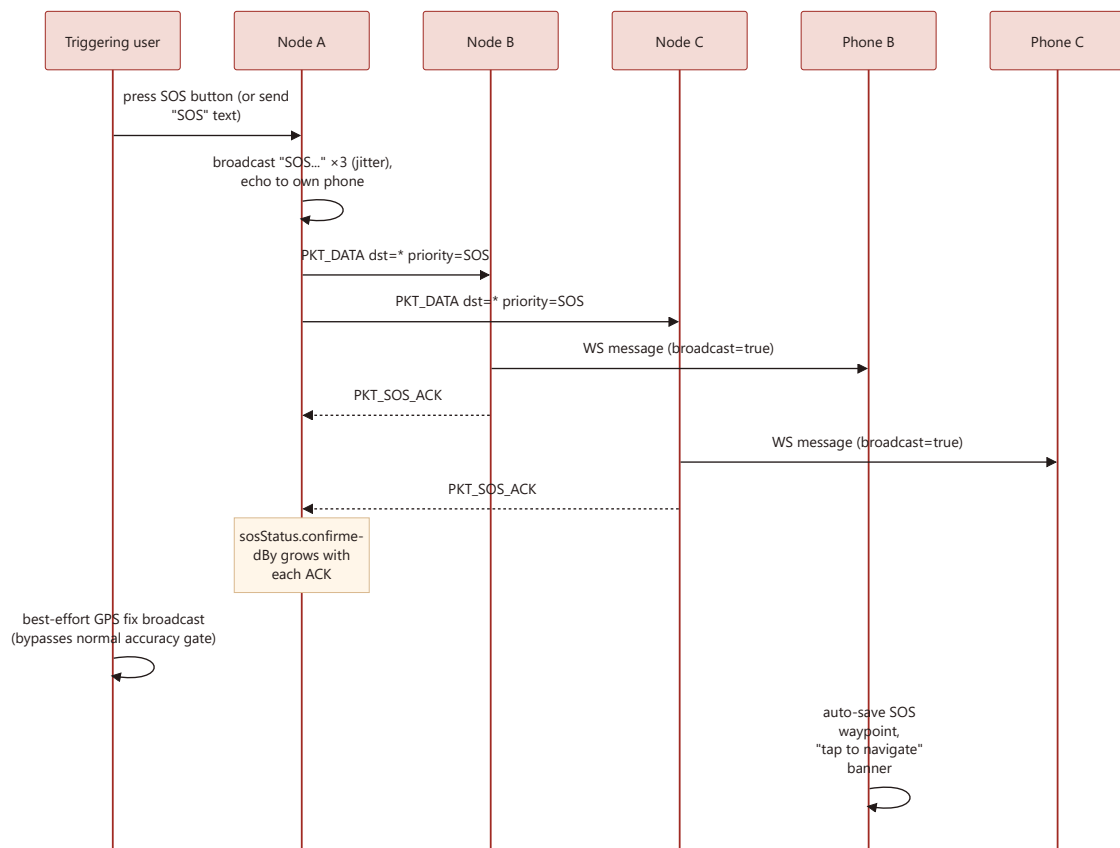


Figure 11.1 — SOS propagation and confirmation. Every receiving node's **PKT_SOS_ACK** is best-effort and unretried by design (Section 5.7) — the goal is aggregate confidence ("N nodes confirmed receipt"), not guaranteed per-recipient delivery, which would require the same costly ACK-storm-prone mechanism used for ordinary unicast traffic.

The confidence banner is the one piece of transient UI in the entire application that never auto-dismisses on a timer — during an actual emergency this is judged the

single most important context a user can have on screen, and is dismissed only by an explicit user action or superseded by a fresh SOS trigger.

Mobile Application Architecture

12.1 Platform Choice — Web-Native Hybrid

DESIGN DECISION

Chosen: a single Next.js/React/TypeScript codebase, statically exported and wrapped by Capacitor into a native Android (and, prospectively, iOS) shell, rather than two separate native codebases or a plain installable web app (PWA) with no native shell at all.

Why not fully native: the application's logic — protocol encoding/decoding, state management, UI — is identical across platforms; maintaining separate Kotlin/Swift implementations of the same mesh protocol and UI would roughly double the surface area for bugs without any capability the target feature set actually needs.

Why not a plain PWA: several capabilities are unavailable or unreliable from a pure browser context on Android — continuous background GPS tracking for live-share (Section 10.2), reliable on-screen-keyboard height reporting (Section 13.1), OS-level network-state awareness distinct from browser `online` / `offline` events (which proved unreliable inside an Android WebView specifically), and app-lifecycle (foreground/background) signals. Capacitor's native plugin bridge (`@capacitor/keyboard`, `@capacitor/network`, `@capacitor/app`, `@capacitor/geolocation` -equivalent browser APIs) closes exactly these gaps while keeping the application code itself entirely web-standard.

12.2 Component and State Architecture

Nearly all application state lives in one top-level orchestrator component, which owns the WebSocket connection, every conversation thread, the contact roster, and every active session (navigation, live-share, SOS, discovery). Presentational views and modals receive state and callbacks as props and contain no independent source of

truth of their own — a deliberate single-owner model that keeps state transitions traceable to one place rather than scattered across a tree of components each holding a fragment of the truth.

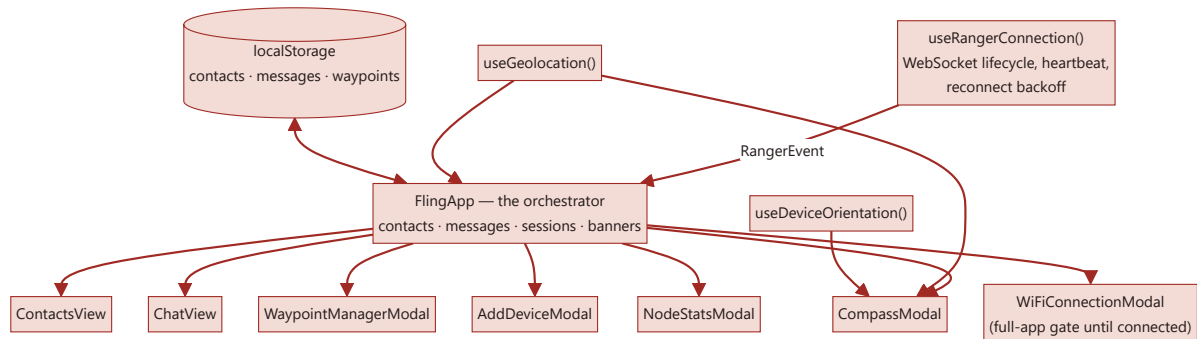


Figure 12.1 — Component and data-flow architecture. Every view is a pure function of state owned by the orchestrator; only two hooks (geolocation, device orientation) maintain their own independent state, and both do so because their data is inherently local to the device rather than part of the mesh session.

12.3 Connection Lifecycle and Resilience

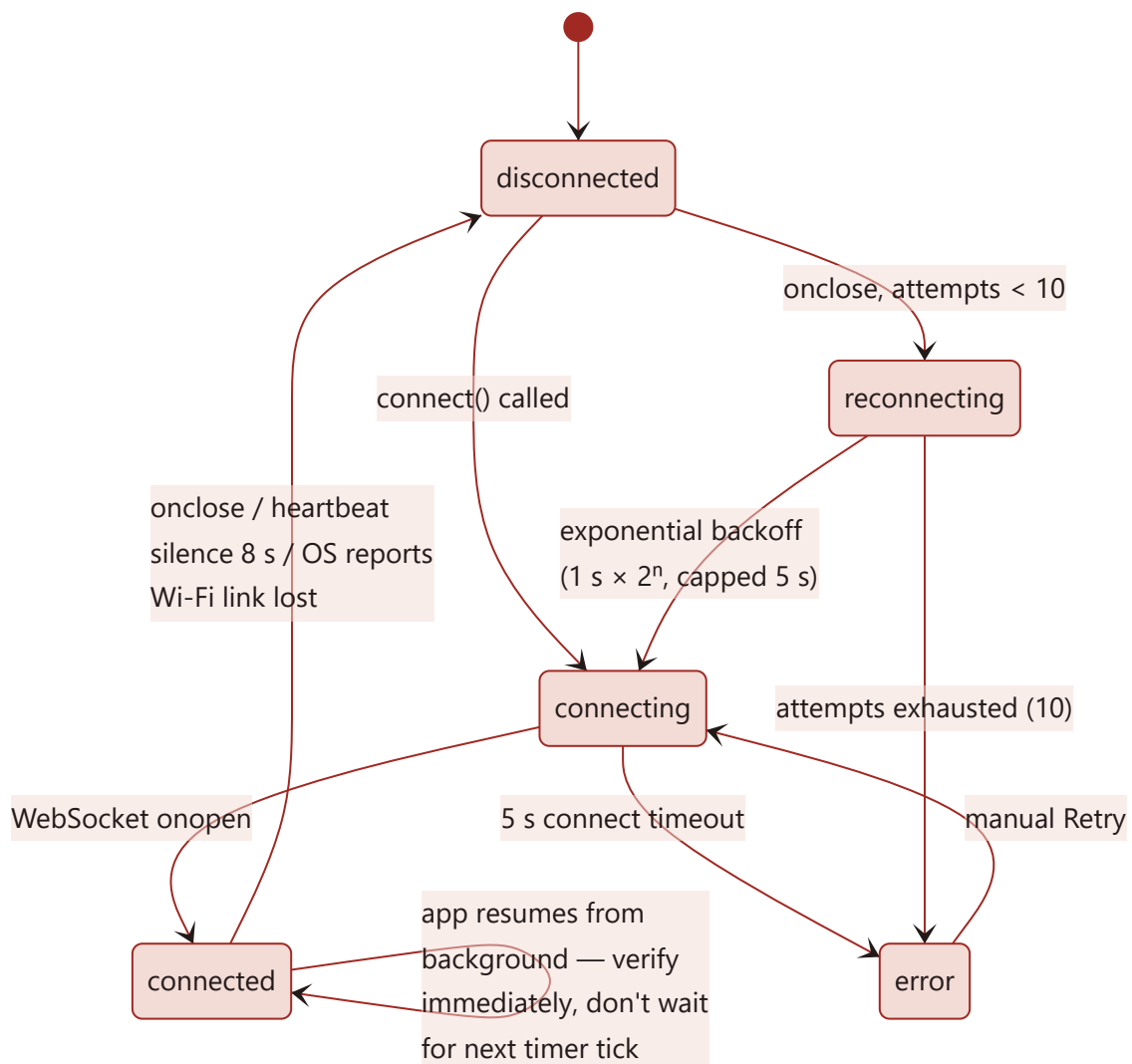


Figure 12.2 — WebSocket connection-state machine. Three independent signals can force a transition out of **connected**: the socket's own close event, the application heartbeat's silence detector (Section 6.5), and the OS network-state listener reacting to a real Wi-Fi loss before the socket has even noticed.

All conversation, contact, and waypoint data persists to **localStorage** as plain JSON, reviving ISO date strings back into **Date** objects on read; the Emergency broadcast pseudo-contact is deliberately excluded from persistence and recreated fresh on every launch, since it represents a channel rather than a real device with a state worth remembering across sessions.

Cross-Cutting Engineering Case Studies

The two problems below are presented in detail because they are representative of a category of bug that is easy to introduce and unusually hard to diagnose from symptoms alone: several independently reasonable pieces of code, each behaving exactly as its own author intended, compensating for the same real-world condition *more than once*. Neither was a logic error in any single function; both were emergent from the interaction between layers that had no visibility into what the other layers were already doing.

13.1 The Triple Keyboard-Compensation Bug

CHALLENGE

Field testing repeatedly surfaced the same complaint about the chat message input: when the on-screen keyboard opened, the input box did not sit flush above it — it floated with a visible gap, or in some cases appeared to overlap it, depending on device and keyboard state.

Diagnosis revealed **three independent mechanisms were all compensating for the same keyboard, simultaneously**, each unaware the others existed:

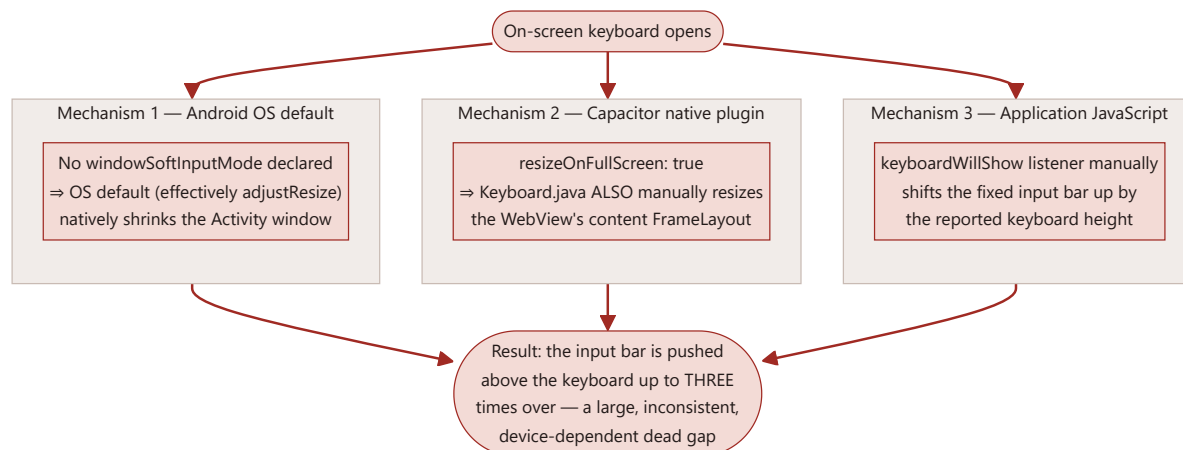


Figure 13.1 — Three uncoordinated compensations for one event. Each mechanism, read in isolation, is a textbook-correct way to keep content above a keyboard; the defect existed only in their combination.

INVESTIGATION PATH

The first fix attempt changed only the Capacitor configuration's `resize` string from `'Body'` to `'None'` — the documented Capacitor knob for this exact scenario — and did not resolve the issue. Reading the plugin's native Android source directly (`Keyboard.java`) revealed why: on Android, that plugin's implementation never actually reads the `resize` string at all; only the separate `resizeOnFullScreen` boolean controls whether it performs its own native resize, and that flag had been left `true` . A second, deeper pass then surfaced Mechanism 1 above — since the Android manifest declared no explicit `windowSoftInputMode` , the OS's own default behaviour was independently resizing the window before either Capacitor or the application's own JavaScript ever ran.

MITIGATION

All three layers now agree explicitly rather than accidentally: `resizeOnFullScreen: false` disables Capacitor's own native resize;

`android:windowSoftInputMode="adjustNothing"` on the main Activity disables the OS's default resize; and the application's own JavaScript — which already had a carefully-tuned, animated, keyboard-height-aware layout (built for exactly this purpose but previously fighting two invisible co-compensators) — is left as the single source of truth for keyboard-avoiding layout. The general lesson generalises beyond this one bug: whenever a platform offers automatic behaviour for a problem the application also intends to solve itself, the automatic behaviour must be verified *disabled*, not merely assumed absent by default — a plugin configuration flag with a plausible-sounding name (`resize: 'None'`) is not proof that the underlying native code actually reads it.

13.2 Notification Stacking and Edge-to-Edge Layout

CHALLENGE

The application accumulated several independent top-anchored transient banners over its development — an SOS confidence banner, a reachability-change notice, a delivery-failure notice, a location-session notice, a live-sharing indicator, and a "resume navigation" pill — each added at a different point in the project and each implemented as its own independently `fixed` -position element. Two of these (the live-share indicator and the navigation resume pill) were positioned at the exact same fixed offset as the banner stack above them, with the visual stacking between the others achieved by hand-computed pixel `marginTop` offsets that assumed a fixed height per banner type. Because an active SOS state and an active navigation session are both especially likely to occur *together* during a real emergency — precisely the scenario this application exists for — the SOS banner could silently render directly on top of, and fully hide, the navigation resume pill underneath it.

MITIGATION

All six elements were consolidated into a single `fixed`, top-anchored flex column, so that stacking order and spacing are now determined by actual rendered content height rather than a guessed constant — a banner whose text happens to wrap to an extra line on a narrow phone now simply pushes the next banner further down, rather than overlapping it. The container itself remains `pointer-events: none` so empty space between banners never blocks taps on the screen beneath; each individual banner re-enables pointer events on itself only if it is actually interactive.

A related, still-latent consideration is Android edge-to-edge layout: the application's minimum and target SDK levels (23 and 35 respectively) mean that on Android 15 and later, the operating system may let the app's content draw underneath the system status bar and gesture-navigation bar by default, regardless of any Capacitor configuration. Fixed-to-bottom interactive elements (the chat input bar, the "add contact" floating action button, modal footer button bars) now carry additive `env(safe-area-inset-bottom)`-aware padding — additive in the sense that it only ever grows beyond the already-tuned baseline padding, never shrinks it, so behaviour is unchanged on the many devices that do not report a meaningful inset while remaining correct on the ones that do.

End-to-End System Walkthrough

Figures 14.1 and 14.2 synthesise the whole system into one continuous scenario, split into two halves for legibility: two nodes discover and pair for the first time and exchange a chat message across a relay (Figure 14.1), then one of them triggers an SOS and the other requests and receives live location (Figure 14.2) — deliberately chaining together the flows detailed individually in Sections 5, 6, 7, 10, and 11, to show how they compose in practice rather than only in isolation.

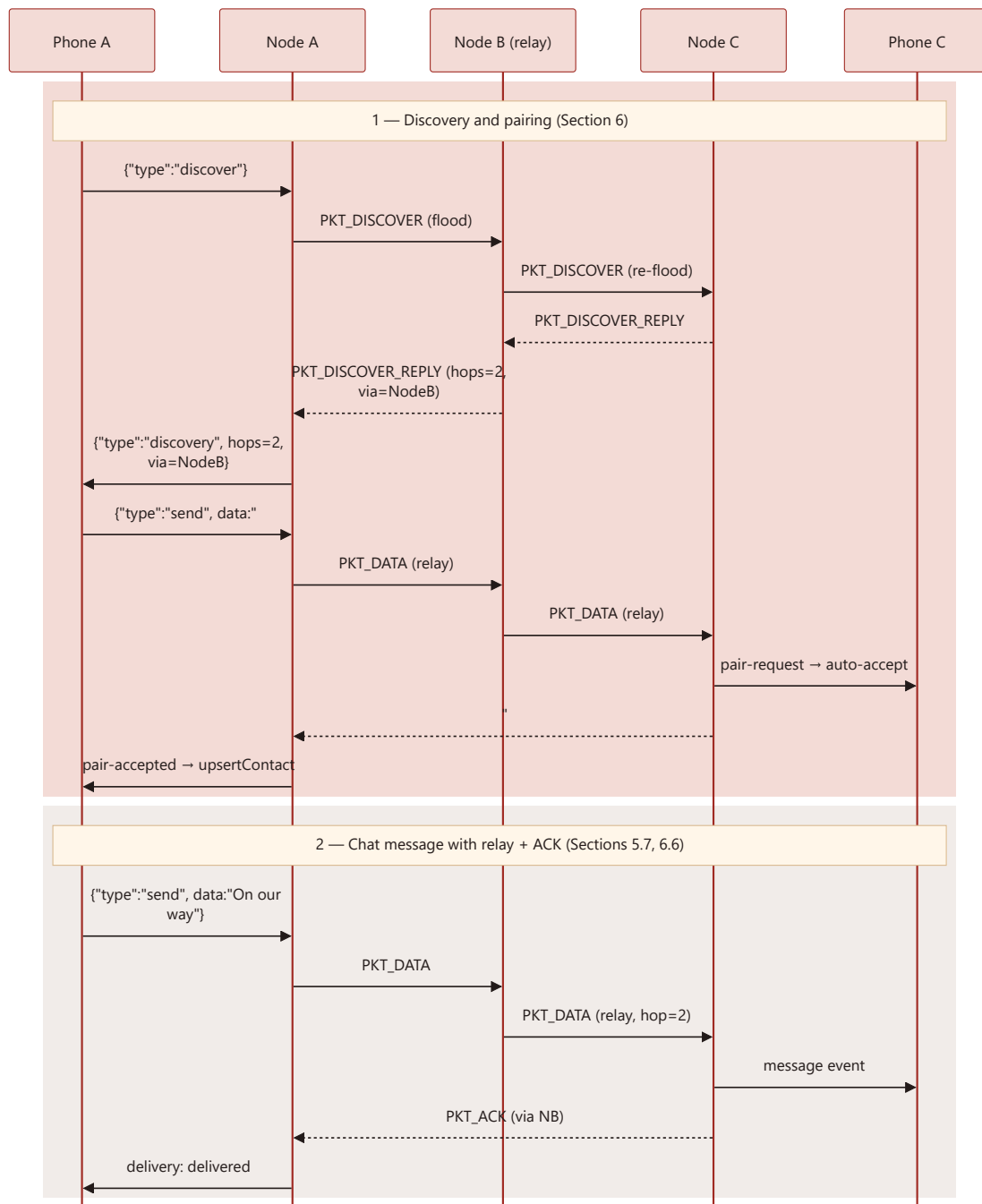


Figure 14.1 — Discovery, pairing, and a relayed chat message across a three-node, two-relay-hop topology (part 1 of 2).

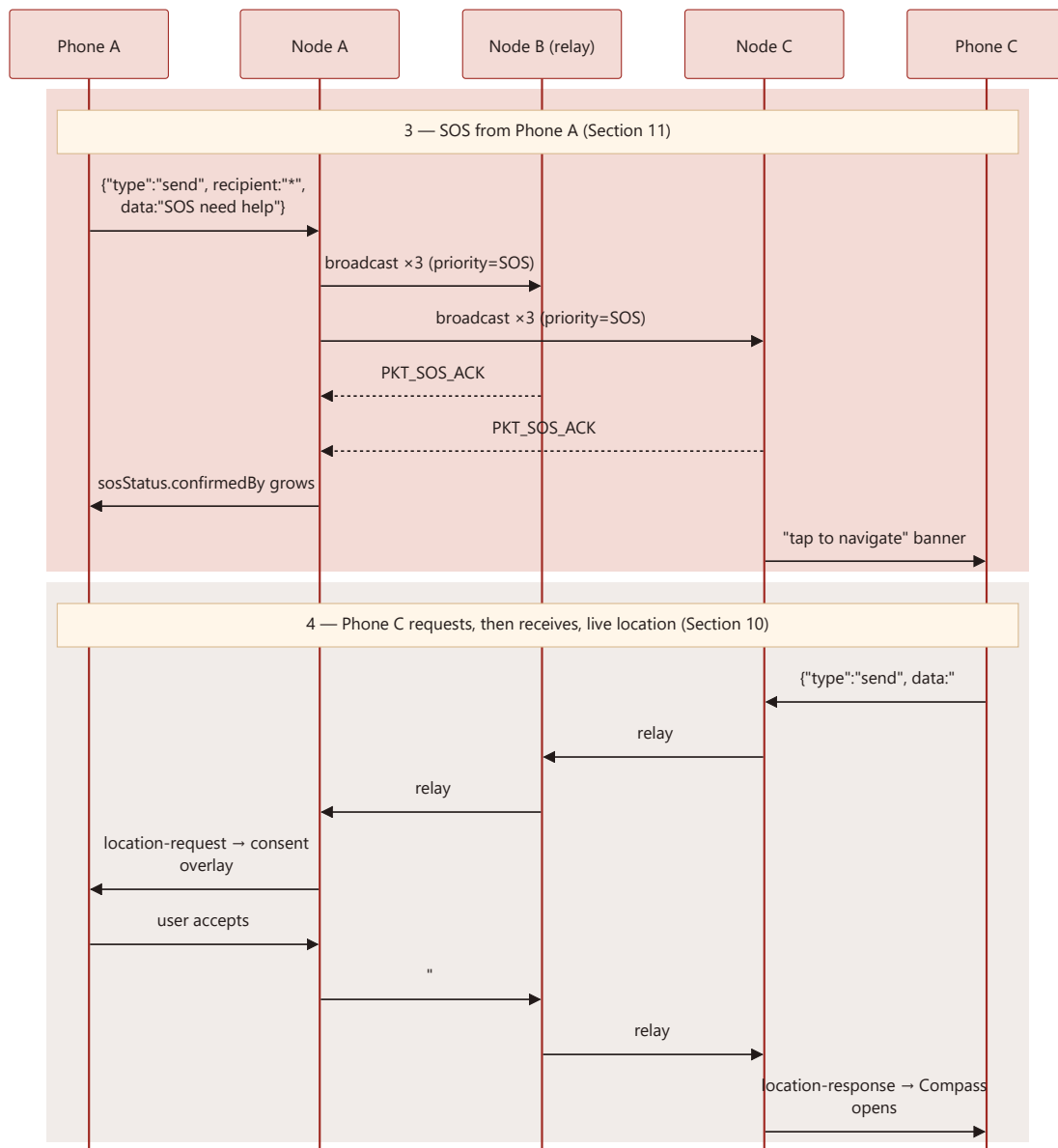


Figure 14.2 — Emergency broadcast and consented live location sharing (part 2 of 2), continuing directly from Figure 14.1.

Design Trade-offs Summary

The following table consolidates every major "path not taken" discussed across this document into a single reference, alongside the concrete reason the chosen alternative fit this project's specific constraints — small mesh scale, severe power/memory budget, guaranteed zero connectivity, and a compressed development timeline.

Subsystem	Alternative considered	Chosen instead	Deciding factor
Firmware concurrency	FreeRTOS multi-task / interrupt-driven	Single-threaded poll loop	Eliminates entire class of shared-state race conditions on constrained hardware
Mesh routing strategy	Proactive (OLSR/RIP-style)	Reactive, AODV-inspired	Airtime is the scarcest resource; only pay routing cost for destinations actually used
Routing robustness	Full AODV (sequence numbers, precursor lists)	Simplified: strict TTL + reactive failure detection	Structural loop-safety at a fraction of the implementation cost, appropriate at this mesh scale
Phone ↔ node transport	HTTP polling / raw byte protocol	Persistent WebSocket + JSON	Push-driven traffic pattern; JSON's forward-compatibility absorbed protocol growth without breakage
Location/pairing semantics	Firmware-level message parsing	App-level text sentinels over opaque payloads	Keeps the firmware protocol-agnostic; new app features need zero firmware changes

Subsystem	Alternative considered	Chosen instead	Deciding factor
Offline navigation	Bundled/downloadable offline map tiles (Leaflet)	Compass bearing-and-distance only	Removed a connectivity-dependent bootstrapping problem from a zero-connectivity system, under time pressure
Compass heading source (Android)	Plain <code>deviceorientation</code> (relative)	<code>deviceorientationabsolute</code> , with relative fallback discarded once absolute is confirmed	Only the absolute event is referenced to true magnetic north
Live-share cadence	Fixed-interval polling timer	Continuous <code>watchPosition</code> + movement/heartbeat decision on every callback	Survives Android's background GPS throttling of repeated cold-start location requests
Mobile app platform	Separate native iOS/Android codebases	Single Next.js/React codebase + Capacitor	One implementation of protocol/state logic; native plugins fill the few genuine platform gaps
Keyboard-avoidance layout	Rely on native OS/plugin auto-resize	Fully manual, JavaScript-driven layout with all native auto-resize explicitly disabled	Automatic behaviours from three independent layers were silently compounding (Section 13.1)

Limitations and Future Work

- **Manual node identity assignment.** Each node's ID and Wi-Fi credentials are set at flash time; nothing in the protocol prevents two accidentally-identical IDs from being deployed, which would be indistinguishable to the mesh. An auto-negotiated or factory-unique identity scheme is a natural extension.
- **Single-phone-per-node pairing.** The Wi-Fi AP model means exactly one phone can be usefully attached to a given node at a time; a busier deployment (e.g. a small team sharing one node) would need either multi-client AP support or a phone-to-phone relay above the existing node-to-node mesh relay.
- **No persistent store-and-forward across power cycles.** The firmware's pending-ACK and pending-route tables are entirely in RAM; a node that reboots mid-delivery loses that in-flight state. Given the emergency-first use case, a message that must survive a node reboot is arguably better served by the sender's own retry-on-reconnect behaviour than by firmware-side durability, but this is an explicit, not accidental, limitation worth stating.
- **No terrain-aware navigation.** The compass-only navigation described in Section 9 is an intentional, informed trade-off (Section 9.1), but it is a real capability gap relative to the map-based mode that was removed — a future iteration with more development time could revisit a lightweight, genuinely offline-bootstrappable map layer without repeating the specific pitfalls documented in Section 9.1.
- **iOS packaging is prospective.** The Capacitor/Next.js architecture targets iOS as a first-class platform in principle (the codebase already branches for `webkitCompassHeading` and iOS permission-prompt flows), but field validation to date has been Android-only.

Conclusion

ATCS demonstrates that a genuinely useful emergency communication system can be built from inexpensive, self-contained hardware and a conventional web application stack, provided the two are bridged by a protocol layer disciplined enough to never assume connectivity it cannot guarantee. The system's mesh routing is deliberately smaller and simpler than textbook AODV, its application protocol deliberately keeps the firmware ignorant of anything beyond addressed byte delivery, and — as documented candidly in Sections 9.1, 13.1, and 13.2 — several subsystems reached their final form only after an earlier, more ambitious version was tried, measured against real field conditions, and consciously simplified. That pattern, more than any single algorithm, is the project's central engineering lesson: in a system whose entire value proposition is working when nothing else does, the simplest mechanism that can be verified correct under real conditions consistently outperformed the more sophisticated one that could only be reasoned about in theory.

Appendix A — Firmware Constants Reference

Complete listing of timing, capacity, and threshold constants defined in `V2_ranger_rola.ino`, for reference alongside Sections 4 and 5.

Constant	Value	Purpose
<code>NODE_ID_LEN</code>	8	Fixed-width address field, bytes
<code>MAX_PAYLOAD</code>	180	Maximum payload bytes per packet
<code>HEADER_SIZE</code>	31	MeshHeader size, bytes
<code>MAX_PACKET_SIZE</code>	211	Header + max payload
<code>MESH_TTL_MAX</code>	6	Maximum hops a packet may travel
<code>MAX_NEIGHBORS</code>	12	Neighbor table capacity
<code>MAX_ROUTES</code>	12	Route table capacity
<code>MAX_SEEN</code>	40	Duplicate-detection cache capacity
<code>MAX_OUTQUEUE</code>	14	Outgoing TX queue capacity
<code>MAX_PENDING_ACK</code>	10	In-flight unicast sends awaiting ACK
<code>MAX_PENDING_ROUTE</code>	4	Sends buffered awaiting route discovery
<code>HELLO_INTERVAL_MS</code>	15,000	Period between HELLO beacon rounds
<code>HELLO_TOTAL_SENDS</code> / <code>HELLO_REPEAT_GAP_MS</code>	3 / 400	Redundant beacon burst size and spacing
<code>NEIGHBOR_EXPIRY_MS</code>	45,000	Forget a neighbor after this long unheard
<code>NEIGHBOR_REPORT_MIN_GAP_MS</code>	3,000	Rate limit on "neighbor" WebSocket frames, per neighbor
<code>ROUTE_EXPIRY_MS</code>	60,000	Forget a route after this long unused
<code>ROUTE_DISCOVERY_TIMEOUT_MS</code>	8,000	Total budget across all RREQ attempts
<code>MAX_RREQ_ATTEMPTS</code>	2	1 initial + 1 controlled retry
<code>RREQ_RETRY_INTERVAL_MS</code>	3,000	Resend RREQ if no RREP within this window
<code>ROUTE_KEEPA_LIVE_MARGIN_MS</code>	15,000	Start proactive refresh this far before route expiry
<code>ROUTE_KEEPA_LIVE_TICK_MS</code>	8,000	Minimum gap between keepalive probes

Constant	Value	Purpose
ACK_TIMEOUT_MS	5,000	Wait time for end-to-end ACK before resend
MAX_SEND_RETRIES	5	Unicast resend attempts before route invalidation
SEEN_CACHE_EXPIRY_MS	30,000	Duplicate-cache entry lifetime
DISCOVER_SCAN_WINDOW_MS	7,000	<code>isScanning</code> auto-clear window
SOS_TOTAL_SENDS / SOS_REPEAT_GAP_MS	3 / 250	Redundant SOS broadcast burst size and spacing
JITTER_UNICAST_MIN/MAX_MS	10 / 60	Random TX delay range, unicast packets
JITTER_FLOOD_MIN/MAX_MS	60 / 300	Random TX delay range, flooded packets
RX_QUIET_GUARD_MS	25	TX hold-off after any packet reception
WebSocket heartbeat	15,000 / 3,000 / ×2	Ping interval / pong timeout / misses before disconnect
Stats broadcast period	5,000	Interval for periodic <code>"stats"</code> frame

All time values in milliseconds unless noted otherwise.

Application-side timing constants

Constant	Value	Purpose
<code>NODE_STALE_MS</code> / <code>NODE_OFFLINE_MS</code>	22,000 / 45,000	Reachability thresholds (Section 8)
<code>RSSI_STRONG_DBM</code> / <code>RSSI_WEAK_DBM</code>	−85 / −100	Signal-quality thresholds (Section 8)
<code>RANGE_CHECK_INTERVAL_MS</code>	5,000	Reachability re-evaluation tick
<code>LOCATION_REQUEST_TIMEOUT_MS</code>	30,000	Give up waiting for a location response
<code>MIN_LOCATION_MOVE_M</code>	8	Movement threshold to trigger a live-share send
<code>LIVE_SHARE_HEARTBEAT_MS</code>	10,000	Send a live-share fix even while stationary
<code>MAX_USEFUL_GPS_ACCURACY_M</code>	100	Fixes worse than this are withheld (except the first send and SOS)
<code>SOS_LOCATION_DEBOUNCE_MS</code>	8,000	Collapse the firmware's 3× SOS resend into one logical trigger
<code>ACK_RETRY_VISUAL_DELAY_MS</code>	5,500	Client-side "sent" → "retrying" visual transition (Figure 7.1)
<code>ARRIVAL_DISTANCE_M</code>	2	Compass "arrived" threshold (Section 9.4)
<code>GPS_WARMUP_GOOD_ACCURACY_M</code> / <code>_MAX_WAIT_MS</code>	30 / 6,000	GPS warm-up window before fixes are trusted for navigation

B

Appendix B — WebSocket JSON Schema Reference

Illustrative example frames for the message catalogue defined in Section 6.

```
// Firmware → Phone, on connect
{ "type": "device_info", "deviceId": "Node4", "deviceName": "ATCS-node 4",
  "frequency": 433000000, "spreadingFactor": 9, "bandwidth": 125000 }

// Firmware → Phone, incoming message (relayed, 2 hops)
{ "type": "message", "sender": "Node1", "data": "Reached the ridge",
  "rssi": -78, "hops": 2, "via": "Node2" }

// Firmware → Phone, broadcast (Emergency channel)
{ "type": "message", "sender": "Node1", "data": "SOS need help",
  "broadcast": true }

// Firmware → Phone, delivery outcomes
{ "type": "delivery", "status": "delivered" }
{ "type": "delivery", "status": "failed", "dest": "Node3" }
{ "type": "delivery", "status": "sos_received", "from": "Node2" }

// Firmware → Phone, periodic diagnostics (every 5 s)
{ "type": "stats", "pktSent": 512, "appMsgSent": 44, "messagesReceived": 39,
  "uptime": 8120, "connectedClients": 1, "pktForwarded": 61,
  "pktDroppedDup": 3, "pktDroppedNoRoute": 0, "pktDroppedQueueFull": 0,
  "routeDiscoveries": 2 }

// Phone → Firmware, send a message
{ "type": "send", "recipient": "Node3", "data": "On our way" }

// Phone → Firmware, beep / discover / ping
{ "type": "beep", "recipient": "Node3" }
{ "type": "discover" }
{ "type": "ping" }

// Application-layer sentinels (carried inside "data" of a "send"/"message" frame)
"##LREQ##"
"##LRESP##7,-6.187222,35.762222,12"
"##LSTOP##"
"##PAIR_REQ##Phone A"
"##PAIR_ACK##Phone C"
```

